UNIWERSYTET
IM. ADAMA MICKIEWICZA
W POZNANIU

Wydział Matematyki i Informatyki

Dominik Jagosz
Numer albumu: 464871

Podział sekretu i obliczenia wielostronne na kryptograficznej platformie aukcyjnej „Aukciszek”

Secret Sharing and Secure Multiparty Computation on the Cryptographic Auction Platform "Aukciszek"

Praca magisterska na kierunku **informatyka**
(specjalność: cyberbezpieczeństwo)
napisana pod opieką
dra Jędrzeja Garnka

Poznań, wrzesień 2025

Poznań, 29 września 2025 r.

Oświadczenie

Ja, niżej podpisany **Dominik Jagosz**, student Wydziału Matematyki i Informatyki Uniwersytetu im. Adama Mickiewicza w Poznaniu oświadczam, że przedkładaną pracę dyplomową pt. *Podział sekretu i obliczenia wielostronne na kryptograficznej platformie aukcyjnej „Aukciszek"* napisałem samodzielnie. Oznacza to, że przy pisaniu pracy, poza niezbędnymi konsultacjami, nie korzystałem z pomocy innych osób, a w szczególności nie zlecałem opracowania rozprawy lub jej części innym osobom, ani nie odpisywałem tej rozprawy lub jej części od innych osób. Oświadczam również, że egzemplarz pracy dyplomowej w wersji drukowanej jest całkowicie zgodny z egzemplarzem pracy dyplomowej w wersji elektronicznej. Jednocześnie przyjmuję do wiadomości, że przypisanie sobie, w pracy dyplomowej, autorstwa istotnego fragmentu lub innych elementów cudzego utworu lub ustalenia naukowego stanowi podstawę stwierdzenia nieważności postępowania w sprawie nadania tytułu zawodowego.

[TAK] wyrażam zgodę na udostępnianie mojej pracy w czytelni Archiwum UAM

[TAK] wyrażam zgodę na udostępnianie mojej pracy w zakresie koniecznym do ochrony mojego prawa do autorstwa lub praw osób trzecich

Streszczenie

W pracy został przedstawiony kryptograficznie zabezpieczony protokół rozstrzygania aukcji, oparty na schemacie podziału sekretu Shamira oraz protokole BGW (Ben-Or - Goldwasser - Wigderson) umożliwiającym wykonywanie operacji arytmetycznych na podzielonych danych w sposób rozproszony i bezpieczny. Kluczowym elementem jest operacja porównania, która pozwala wyłonić najkorzystniejszą ofertę bez ujawniania różnic pomiędzy poszczególnymi ofertami. Zaimplementowany protokół stanowi przykład zastosowania metod kryptografii wielostronnej (MPC) w scenariuszu aukcyjnym i został opracowany w ramach projektu „Aukciszek".

Słowa kluczowe: aukcja, kryptografia, obliczenia wielostronne, podział sekretu

Abstract

This thesis presents the cryptographically secure auction system, implementing Shamir's secret sharing scheme and the BGW (Ben-Or - Goldwasser - Wigderson) protocol for secure distributed arithmetic operations on shared data. The key element of the system is the comparison protocol, which selects the best offer without revealing the difference between offered values. It was developed for the "Aukciszek" platform as an example of multi-party computation application in an auction scenario.

Keywords: auction, cryptography, multi-party computation, secret sharing

Spis treści

Rozdział 1. Założenia projektu . . . 7
1.1. Motywacja . . . 7
1.1.1. Założenia . . . 8
1.1.2. Rozwiązanie . . . 8
1.1.3. Cel pracy . . . 9
Rozdział 2. Podział sekretu i operacje arytmetyczne . . . 10
2.1. Podział sekretu . . . 10
2.1.1. Schematy podziału sekretu . . . 10
2.1.2. Schemat Shamira . . . 13
2.2. Operacje na podzielonych sekretach . . . 18
2.2.1. Mnożenie wielomianu przez stałą . . . 19
2.2.2. Dodawanie wielomianów . . . 20
2.2.3. Wykrycie i naprawa błędów rekonstrukcji sekretu . . . 21
2.3. Mnożenie sekretów . . . 28
2.3.1. Redukcja stopnia wielomianu . . . 29
2.3.2. Zapewnienie losowości współczynników wielomianu . . . 31
2.3.3. Mnożenie wielomianów . . . 32
2.3.4. Ograniczenie liczby rund komunikacji . . . 36
2.3.5. Złożoność obliczeniowa mnożenia . . . 38
Rozdział 3. Porównanie sekretów . . . 39
3.1. Porównanie liczb na platformie Aukciszek . . . 39
3.1.1. Bezpieczny XOR . . . 40
3.1.2. Podział losowego bitu . . . 40
3.1.3. Analiza znaku różnicy sekretów . . . 42
3.1.4. Wyznaczenie wartości przeniesienia . . . 42
3.1.5. Porównanie liczb na podstawie ich podziałów . . . 44
3.1.6. Równoległe wykonywanie wielu rund komunikacji . . . 45
3.1.7. Bezpieczeństwo . . . 46
3.2. Porównanie liczb na podstawie podziałów ich bitów . . . 47
3.2.1. Podział losowej liczby z $\mathbb{Z}_p$ . . . 48
3.2.2. Porównanie liczb na podstawie podziałów ich bitów . . . 49

3.2.3. Wyznaczenie podziałów bitów liczby 51
Rozdział 4. Implementacja projektu 55
4.1. Implementacja systemu aukcyjnego 55
4.1.1. Strony aukcji 55
4.1.2. Składanie ofert 56
4.1.3. Rozstrzygnięcie aukcji 56
4.1.4. Porównanie ofert 57
4.1.5. Ogłoszenie wygrywającej oferty 58
4.2. Testy i zakończenie projektu 58

Rozdział 1

Założenia projektu

1.1. Motywacja

Celem pracy jest przedstawienie podstaw teoretycznych, stojących za działaniem kryptograficznie bezpiecznej platformy aukcyjnej *Aukciszek*. Powstała ona w ramach projektu badawczo-rozwojowego na Wydziale Matematyki i Informatyki Uniwersytetu im. Adama Mickiewicza w Poznaniu, w roku akademickim 2024/2025. W skład zespołu projektowego wchodzili studenci informatycznych studiów magisterskich: Franciszek Czajka, Karol Filipiak, Dominik Jagosz (autor niniejszej pracy), Mateusz Piwowarski. Ze strony kadry wydziału, opiekę nad pracami sprawowali: prof. UAM dr hab. Maciej Grześkowiak, dr Jędrzej Garnek, dr Jolanta Marzec-Ballesteros. Celem projektu było stworzenie platformy aukcyjnej zachowującej maksymalną poufność ofert. Prace zostały zakończone pomyślnie, a kod otwartoźródłowego rozwiązania jest dostępny do wglądu pod adresem github.com/Aukciszek.

Motywacją zespołu było przekonanie, że użytkownicy sytemu aukcyjnego zyskują na dobrym zabezpieczeniu tajemnicy ich ofert. W przetargach i negocjacjach dotyczących wysokiej wartości dóbr lub usług, kluczowe jest zabezpieczenie przed wyciekiem informacji, gdyż znajomość wysokości ofert innych uczestników może dać oferentowi znaczącą przewagę podczas aukcji. Chcieliśmy wykorzystać oparte na matematyce metody kryptograficzne do stworzenia przejrzystego protokołu, który zasługiwałby na wysokie zaufanie użytkowników. Mieliśmy nadzieję, że pewność używanych narzędzi pozwoli uczestnikom zachowywać się zgodnie z założeniami systemu. Oznaczałoby to, że zainteresowane aukcją podmioty byłyby bardziej skłonne do wzięcia w niej udziału, a rzadziej podejmowałyby próby zabezpieczenia swoich interesów przez obejście systemu.

Aukciszek służy do przeprowadzania aukcji, w której każdy z uczestników składa jedną ofertę, która pozostaje niezmienna do momentu rozstrzygnięcia.

Tożsamości stron składających oferty są znane, ale wysokości ofert są tajne. Jedynym czynnikiem decydującym o wyborze wygrywającej oferty jest cena. Porównanie ofert nie zdradza ich wysokości, pozwala jedynie je uszeregować. Po rozstrzygnięciu aukcji, detale wygrywającej oferty i tożsamość składającego ją uczestnika są ujawniane wszystkim stronom. Wysokości pozostałych ofert pozostają tajne.

Możliwe zastosowania *Aukciszka* to wszelkie aukcje, przetargi i zapytania ofertowe, w ramach których uczestnicy składają jedną tajną ofertę. Możliwe jest też użycie systemu do poufnego porównania danych liczbowych między stronami, służące wyborowi pewnych standardów lub innych ustaleń.

1.1.1. Założenia

W celu zabezpieczenia procesu aukcyjnego, w systemie *Aukciszek* spełnione są następujące założenia:

1. Administrator aukcji ma kontrolę nad dopuszczeniem uczestników do składania ofert.
2. Uczestnicy asynchronicznie składają tajne oferty.
3. Nie jest możliwa modyfikacja złożonej oferty.
4. Porównanie ofert jest przeprowadzane w sposób niezdradzający ich wysokości. Protokół porównania jest znany. Wiadomo, który krok obliczeń jest wykonywany w danym momencie, ale nie można stwierdzić, jakie argumenty są używane.
5. Wszystkie przegrywające oferty pozostają tajne.
6. Wygrywająca oferta jest ujawniana wszystkim stronom.

1.1.2. Rozwiązanie

Aby spełnić założenia, zdecydowaliśmy się na wykorzystanie modelu obliczeń wielostronnych. Pozwala on na utajnienie danych, jednocześnie pozostawiając możliwość obliczenia wartości dowolnej ich funkcji.

Zwykle uczestnicy aukcji muszą zaufać jednemu pośrednikowi, który poznaje wszystkie oferty. Z pomocą *Aukciszka* mogą podzielić swoje zaufanie między kilka częściowo zaufanych podmiotów, które, mimo braku pełnej informacji o ofertach, mogą razem wybrać najlepszą z nich.

Przeprowadzenie aukcji zostaje zatem powierzone grupie podmiotów, której skład jest akceptowalny dla wszystkich stron. Mogą wśród nich być np. rządowe instytucje, prywatne firmy, przedstawiciele uczestników. W przeciwieństwie do sytuacji, gdy aukcja jest nadzorowana przez jednego pośrednika, żadnemu z nich nie trzeba ufać całkowicie.

Zakładamy, że każdy z podmiotów przeprowadzających aukcję będzie wykonywać protokół aukcyjny zgodnie z założeniami i nie przerwie go. W przeciwnym wypadku łatwo może dojść do blokowania aukcji. Mimo że pośrednicy mogą nie dopuścić do rozstrzygnięcia aukcji, nie ułatwia im to poznania wysokości ofert uczestników. Jest to możliwe jedynie przy współpracy ponad połowy z nich. Zakładamy jednak, że strony są na tyle niezależne od siebie, że nie występuje zmowa obejmująca większości z nich.

1.1.3. Cel pracy

W ramach projektu wykonaliśmy system komputerowy, w którym od podstaw zaimplementowaliśmy metody kryptograficzne obliczeń wielostronnych. Obejmowały one schemat podziału sekretu, pozwalający na utajenie ofert, oraz metody bezpiecznego obliczania funkcji podzielonych danych. Zaadaptowaliśmy również odpowiedni protokół porównania ofert, niewyjawiający informacji o analizowanych wartościach. Celem niniejszej pracy jest opis użytych metod kryptograficznych i stojących za nimi idei. Szczególna uwaga jest poświęcona kluczowej operacji porównania liczb.

Rozdział 2

Podział sekretu i operacje arytmetyczne

2.1. Podział sekretu

2.1.1. Schematy podziału sekretu

Celem przedstawionych w tej części protokołów jest udostępnienie sekretnej wartości kilku stronom tak, aby mogły z niej korzystać tylko wspólnie. Możliwe zastosowanie to przechowywanie kluczy do ważnych aktywów, które powinna nadzorować więcej niż jedna osoba, lub udostępnienie pewnych informacji grupie, której można zaufać jako całości, ale niekoniecznie każdemu jej członkowi z osobna. Na podstawie sekretu oblicza się dane nazywane jego udziałami. Znajomość odpowiedniej liczby udziałów pozwala na wyznaczenie sekretu.

Przykład 2.1. Dostęp do teczki z kodami pozwalającymi na użycie broni jądrowej jest dozwolony parze: prezydent - generał, albo (w przypadku nagłej niedostępności prezydenta) trójce: wiceprezydent - sekretarz stanu - generał. Teczka ma trzy zamki A, B, C. Generał jako jedyny posiada klucz do zamka A, wiceprezydent klucz do zamka B, sekretarz stanu klucz do zamka C. Prezydent otrzymuje klucze do zamków B i C. Rozwiązanie używa trzech zamków i pięciu kluczy [Sma16, (19.1)].

Analogicznie można podzielić sekret będący ciągiem bitowym [Sma16, (19.2.2)].

Przykład 2.2. Dostęp do sekretu jest warunkowany jak w przykładzie 2.1. Sekret jest ciągiem bitowym $s \in \{0,1\}^n$. Losowo dobierane są trzy ciągi bitowe $s_i \in \{0,1\}^n$, spełniające zależność $s = s_1 \oplus s_2 \oplus s_3$, gdzie $\oplus$ oznacza operację XOR. Wtedy stronom przydziela się odpowiednie udziały sekretu:

— prezydent: s_1, s_2,

— generał: s_3,

— wiceprezydent: s_1,
— sekretarz stanu: s_2.

Upoważniona para (lub trójka) odzyskuje sekret przez wykonanie operacji XOR na swoich udziałach.

Podobnie można zbudować model podziału sekretu z progiem. Wyróżnia się on tym, że upoważniona do wyznaczenia sekretu jest każda grupa posiadająca dowolne t spośród n udziałów. Wartość t nazywa się progiem rekonstrukcji sekretu [Sma16, (19.2.2)].

Przykład 2.3. Chcemy, by spośród trzech stron P_1, P_2, P_3, dowolne dwie z nich mogły odtworzyć sekret. Sekret jest ciągiem bitowym $s \in \{0,1\}^n$. Losowo dobierane są trzy ciągi bitowe $s_i \in \{0,1\}^n$, spełniające zależność $s = s_1 \oplus s_2 \oplus s_3$. Wtedy stronom przydziela się odpowiednie udziały sekretu:

— $P_1 : s_1, s_2$,
— $P_2 : s_1, s_3$,
— $P_3 : s_3, s_2$.

Powyższa metoda jest prosta, ale wymusza na stronach przechowywanie dużej ilości informacji. Już w przykładzie 2.3 rozmiar udziału jest dwa razy większy od rozmiaru sekretu.

Przykład 2.4. Jedenastu naukowców pracuje nad tajnym projektem. Przechowują dokumenty w szafce, która może być otwarta tylko w obecności co najmniej sześciu z nich. Wtedy najmniejsza liczba zamków, na którą musi być zamykana szafka, to 462. Każdy z naukowców musi nosić co najmniej 252 klucze [Sha79].

Dowód. Dla każdego podzbioru 5 naukowców z 11 istnieje zamek, do którego żaden z nich nie ma klucza i nie może go otworzyć. Nie istnieje żaden inny podzbiór 5 naukowców, który nie może otworzyć tego samego zamka. Jeżeli istniałby dwa podzbiory 5 naukowców, które nie mogą otworzyć tego samego zamka, suma ich obu nie mogłaby otworzyć szafki. Ale jeżeli te podzbiory byłyby różne, oznaczałoby to, że istnieje podzbiór 6 naukowców z 11, który nie może otworzyć szafki, co jest sprzeczne z założeniem. Dlatego najmniejsza liczba zamków, na które musi być zamykana szafka, to liczba podzbiorów 5-elementowych z 11-elementowego zbioru, tzn. $\binom{11}{5} = 462$.

Każdy naukowiec potrzebuje klucza otwierającego zamek, którego nie może otworzyć dowolny podzbiór 5 innych naukowców spośród pozostałych 10. Każdy klucz jet unikalny, ponieważ każdy rozpatrywany podzbiór nie może otworzyć innego zamka. Dlatego najmniejsza liczba kluczy, które musi nosić każdy z naukowców, to liczba podzbiorów 5-elementowych z 10-elementowego zbioru, tzn. $\binom{10}{5} = 252$. □

Przed opisaniem efektywniejszego rozwiązania, podajemy ogólny model schematu podziału sekretu, a także właściwości, jakie powinien spełniać, na podstawie książki [BS23, (22)].

Definicja 2.1. Schemat podziału sekretu z progiem, pozwalający użytkownikowi na podzielenie sekretu na n udziałów, gdzie znajomość dowolnych t z nich pozwala wyznaczyć sekret, składa się z poniższych części [BS23, (22.1)]:

— *Alicja* - osoba chcąca podzielić sekret,
— Z - grupa skończona,
— $s \in Z$ - sekret Alicji,
— $P_1, \ldots, P_n$ - strony, pomiędzy które dzielony jest sekret,
— $G(s, n, t)$ - probabilistyczny algorytm podziału sekretu,
— $[s] = \{s_1, \ldots, s_n\}$ - zbiór udziałów sekretu wygenerowany przez $G(s, n, t)$,
— $J \subseteq \{1, \ldots, n\}$, $|J| = t$ - zbiór indeksów udziałów używanych do rekonstrukcji sekretu,
— $s \leftarrow C(J, \{s_j\}_{j \in J})$ - deterministyczny algorytm rekonstrukcji sekretu.

Zgodnie z [BS23, (2.2.1)] przyjmujemy oznaczenie $c \xleftarrow{R} E(k, m)$. Kiedy E jest algorytmem probabilistycznym, oznacza ono, że zmienna c przyjmuje losowo jedną z możliwych wartości zwracanych przez E na podstawie parametrów k, m. Zapis $k \xleftarrow{R} \mathcal{K}$ oznacza losowy wybór wartości zmiennej k ze zbioru $\mathcal{K}$, o rozkładzie jednostajnym.

Przedstawimy teraz założenia, które powinien spełniać schemat podziału sekretu.

Poprawność [BS23, (Definition 22.1.)]

Zakładamy, że dla każdego $s \in Z$ i każdego poprawnego $[s] \xleftarrow{R} G(s, n, t)$, dla każdego $J \subseteq \{1, \ldots, n\}, |J| = t$ zachodzi $C(J, \{s_j\}_{j \in J}) = s$. Oznacza to, że używając dowolnego zbioru t udziałów, można prawidłowo zrekonstruować sekret z prawdopodobieństwem równym 1.

Bezpieczeństwo [BS23, (Definition 22.2.)]

Niech $[s] \xleftarrow{R} G(s, n, t)$, $[s'] \xleftarrow{R} G(s', n, t)$ będą zbiorami udziałów wygenerowanymi na podstawie s, s'.

Niech $G(s, n, t)[J]$ oznacza podzbiór udziałów $\{s_j\}_{j \in J}$ dla $J \subseteq \{1, \ldots, n\}$. Schemat podziału sekretu jest bezpieczny, jeśli dla wszystkich dopuszczalnych wartości $0 < t \leqslant n$, dla każdego $s, s' \in Z$ i każdego $J \subseteq \{1, \ldots, n\}$, $|J| = t - 1$, rozkład wartości $G(s, n, t)[J]$ jest nierozróżnialny od rozkładu $G(s', n, t)[J]$.

Oznacza to, że używając dowolnego zbioru $t - 1$ udziałów, nie można dowiedzieć się niczego o sekrecie ani przybliżyć jego wartości, nawet dysponując nieograniczoną mocą obliczeniową.

2.1.2. Schemat Shamira

Opublikowany w 1979 r. przez Adi Shamira, popularny schemat podziału sekretu, jest częścią wielu bardziej rozbudowanych protokołów kryptograficznych. Następujący opis bazuje na wprowadzającej go pracy [Sha79] oraz książkach [BS23, (22.1)] i [VMV96, (12.7)].

Sekret jest zakodowany jako liczba z ciała $\mathbb{F}_p$. Strona dzieląca sekret konstruuje wielomian $f(x) \in \mathbb{F}_p[X]$, którego wartość w punkcie zero odpowiada wartości sekretu. Udziały sekretu to współrzędne punktów należących do wielomianu postaci $(x, f(x))$. Na ich podstawie można wyznaczyć wzór $f(x)$ i obliczyć wartość $f(0) = s$.

Liczba pierwsza p jest publicznie znana i musi być większa od używanych wartości n i s. Nie jest wymagana specjalna postać liczby pierwszej, bezpieczeństwo schematu nie opiera się na niej. W algorytmie korzysta się z pierścienia reszt z dzielenia przez p, który z operacją mnożenia modulo p jest ciałem. W szczególności spełniony jest warunek:

$$(a, b \in \mathbb{F}_p,\ a \cdot b \equiv 0\ (mod\ p)) \Leftrightarrow (a = 0\ \vee\ b = 0).$$

W ciele nie ma zatem dzielników zera - elementów a, dla których istnieje niezerowy element b, spełniający $a \cdot b \equiv 0$. Ich występowanie wprowadzałoby niejednoznaczność i uniemożliwiało rekonstrukcję sekretu. Dlatego liczba p nie może być podzielna przez żadną liczbę poza nią samą i 1.

Przykład 2.5. W pierścieniu $\mathbb{Z}_8$ punkty $(1, 6), (5, 6)$ należą zarówno do wykresu wielomianu $f(x) = 2x + 4 \in \mathbb{Z}_8[X]$, jak i do $g(x) = 4x + 2 \in \mathbb{Z}_8[X]$. Wielomiany f, g kodują różne sekrety ($f(0) = 4, g(0) = 2$). To oznacza, że t punktów nie wyznacza jednoznacznie wielomianu stopnia $t - 1$ nad pierścieniem niebędącym ciałem.

Wielomian, na podstawie którego wyznacza się udziały, jest postaci $f(x) = a_0 + a_1x + \cdots + a_{t-1}x^{t-1}$. Wartość wyrazu wolnego a_0 ustala się na równą s, gdzie s jest dzielonym sekretem. Współczynniki $a_1, \ldots, a_{t-1}$ to losowe liczby z $\mathbb{F}_p$. Stopień wielomianu to co najwyżej $t - 1$, gdzie t to liczba udziałów wymagana do odtworzenia sekretu.

Protokół 2.1. Podział sekretu za pomocą schematu Shamira.
Dane:
- *Alicja* - osoba chcąca podzielić sekret,
- $P_1, \ldots, P_n$ - strony, pomiędzy które dzielony jest sekret,
- p - publicznie znana liczba pierwsza,
- t - minimalna liczba udziałów pozwalająca na odtworzenie sekretu,

— $s \in \mathbb{F}_p$ - sekret Alicji.

Przebieg:

1. Alicja dobiera współczynniki wielomianu $f \in \mathbb{F}_p[X]$ stopnia $t-1$, który spełnia warunek $f(0) = s$:
   — ustala $a_0 = s$,
   — losuje $a_1, \ldots, a_{t-1} \xleftarrow{R} \mathbb{F}_p$.
2. Strony ustalają publicznie znane wartości $x_1, \ldots, x_n$, przypisane dla $P_1, \ldots, P_n$. Są to współrzędne x punktów będących ich udziałami, stąd $x \in \mathbb{Z}, x \neq 0$. Zwykle przyjmuje się $x_i = i$, dla $i = 1, \ldots, n$.
3. Dla każdej strony protokołu P_i, Alicja oblicza udział $(x_i, f(x_i))$ i przekazuje go stronie bezpiecznym kanałem.

Wyjście:

— Udziały $s_1 = (x_1, f(x_1)), \ldots, s_n = (x_n, f(x_n))$ stron $P_1, \ldots, P_n$.

W kontekście schematu Shamira, udziałem s_i będziemy nazywać punkt $(x_i, f(x_i))$ lub liczbę $f(x_i)$, zakładając, że odpowiadająca jej wartość x_i jest znana.

Liczba udziałów odpowiada liczbie stron i może być większa niż stopień wielomianu. Znajomość przynajmniej t udziałów pozwala na wyznaczenie wzoru wielomianu, jako że t punktów o różnych współrzędnych x jednoznacznie wyznacza przechodzący przez nie wielomian stopnia $t-1$. Co więcej, obliczenie wartości $f(0)$ nie wymaga wyznaczania wzoru $f(x)$, jako że może być ona obliczona używając interpolacji Lagrange'a.

Interpolacja Lagrange'a [VMV96, (12.71)]

Przez t różnych punktów (x, y) przechodzi dokładnie jeden wielomian stopnia $t-1$. Przykładowo: dwa punkty na płaszczyźnie jednoznacznie wyznaczają przechodzącą przez nie prostą - wielomian stopnia 1. W zadaniach interpolacyjnych ten fakt służy do przybliżania wzoru funkcji danej przez punkty i jej wartości pomiędzy nimi. W schemacie Shamira wielomian wyznaczany przez interpolację Lagrange'a musi być dokładaną rekonstrukcją funkcji wybranej przez stronę dzielącą sekret.

Do rekonstrukcji sekretu używany jest podzbiór $\{s_j\}_{j \in J}$, liczący przynajmniej t udziałów, o indeksach ze zbioru $J \subseteq \{1, \ldots, n\}$. Wartość sekretu, czyli wartość funkcji w punkcie zero, można obliczyć za pomocą postaci Lagrange'a wielomianu

$$f(x) = \sum_{j \in J} y_j L_j(x),$$

gdzie:

$$L_j(x) = \prod_{k \in J \setminus \{j\}} \frac{x - x_k}{x_j - x_k}.$$

Możliwe jest dzielenie przez niezerowe elementy (tzn. mnożenie przez odwrotność), ponieważ każdy niezerowy element z $\mathbb{F}_p$ posiada element odwrotny.

Jako że szukane jest tylko $s = f(0)$, za x można podstawić 0:

$$f(0) = \sum_{j \in J} y_j L_j,$$

$$L_j = \prod_{k \in J \setminus \{j\}} \frac{-x_k}{x_j - x_k} = \prod_{k \in J \setminus \{j\}} \frac{x_k}{x_k - x_j}.$$

Wyrażenia L_j zawierają jedynie publicznie znane współczynniki x_j, dlatego każda strona może obliczyć je zawczasu i niezależnie. Wartości L_j zależą od indeksów wybranych udziałów. Ustalając zbiór stron biorących udział w rekonstrukcji, można obliczyć stałe L_j, a użyć ich później. Po zgromadzeniu dostatecznie wielu wartości $y_j = f(x_j)$ wystarczy przemnożyć je przez stałe L_j i zsumować:

$$s = \sum_{j \in J} f(x_j) L_j.$$

Przykład 2.6. Podział sekretu za pomocą schematu Shamira.
Dane:
— $n = 5$, $t = 2$, $p = 23$,
— $s = 3$.
Podział sekretu:
1. $a_0 = s = 3$,
2. $a_1 = 20 \xleftarrow{R} \mathbb{F}_{23}$,
3. $x_1, \ldots, x_5 = 1, \ldots, 5$,
4. $f(1) = 0, f(2) = 20, f(3) = 17, f(4) = 14, f(5) = 11$.

Przykładowa rekonstrukcja sekretu (zbiór wybrany do rekonstrukcji to $J = \{2, 4\}$):
1. Obliczenie L_i:
— $L_2 = \frac{4}{4-2} = 4 \cdot 2^{-1} = 4 \cdot 12 = 48 \equiv 2 \ (mod\ 23)$,
— $L_4 = \frac{2}{2-4} = 2 \cdot (-2)^{-1} = 2 \cdot 21^{-1} = 2 \cdot 11 \equiv 22 \ (mod\ 23)$.
2. Zebranie udziałów (x_j, y_j) stron $\{S_j\}_{j \in J}$.
3. Obliczenie sekretnej wartości:
$s = f(2) \cdot L_2 + f(4) \cdot L_4 = 20 \cdot 2 + 14 \cdot 22 = 40 + 308 = 348 \equiv 3 \ (mod\ 23)$.

Inna możliwa rekonstrukcja sekretu (zbiór wybrany do rekonstrukcji to $J = \{1, 3, 4, 5\}$):
1. Obliczenie L_i:
— $L_1 = \frac{3}{3-1} \cdot \frac{4}{4-1} \cdot \frac{5}{5-1} = \frac{3}{2} \cdot \frac{4}{3} \cdot \frac{5}{4} = \frac{5}{2} = 5 \cdot 2^{-1} = 5 \cdot 12 = 60 \equiv 14 \ (mod\ 23)$,
— $L_3 = \frac{1}{1-3} \cdot \frac{4}{4-3} \cdot \frac{5}{5-3} = \frac{1}{-2} \cdot \frac{4}{1} \cdot \frac{5}{2} = -5 \equiv 18 \ (mod\ 23)$,
— $L_4 = \frac{1}{1-4} \cdot \frac{3}{3-4} \cdot \frac{5}{5-4} = \frac{1}{-3} \cdot \frac{3}{-1} \cdot \frac{5}{1} \equiv 5 \ (mod\ 23)$,
— $L_5 = \frac{1}{1-5} \cdot \frac{3}{3-5} \cdot \frac{4}{4-5} = \frac{1}{-4} \cdot \frac{3}{-2} \cdot \frac{4}{-1} = -\frac{3}{2} = -3 \cdot 2^{-1} = 20 \cdot 12 = 240 \equiv 10 \ (mod\ 23)$.

2. Zebranie udziałów (x_j, y_j) stron $\{S_j\}_{j \in J}$.
3. Obliczenie sekretnej wartości:
   $s = f(1) \cdot L_1 + f(3) \cdot L_3 + f(4) \cdot L_4 + f(5) \cdot L_5$
   $s = 0 \cdot 14 + 17 \cdot 18 + 14 \cdot 5 + 11 \cdot 10 = 0 + 306 + 70 + 110 = 486 \equiv 3 \ (mod\ 23)$.

Używając co najmniej dwóch udziałów, można również wyznaczyć dokładny wzór wielomianu $f(x)$, jednak nie jest to potrzebne do poznania sekretu.

Złożoność obliczeniowa schematu Shamira

Kiedy stopień wielomianu to $t-1$, złożoność obliczenia jego wartości w danym punkcie wynosi $O(t-1)$. Strona dzieląca sekret w celu wygenerowania udziałów musi wyznaczyć n punktów. Dlatego złożoność podziału sekretu za pomocą schematu Shamira to $O(n \cdot t)$.

Rekonstrukcja sekretu przez jedną stronę wymaga obliczenia wyrażeń $L_j, j \in J$ i przemnożenia ich przez otrzymane $|J|$ udziałów. Stała L_j to iloczyn $|J|-1$ składników. Z definicji $|J|$ ma co najwyżej n elementów. Stąd złożoność rekonstrukcji sekretu wynosi $O(n^2)$.[1]

Przedstawiona złożoność odpowiada liczbie operacji w $\mathbb{F}_p$ - dodawania, odejmowania, mnożenia lub dzielenia. Dzielenie realizowane jest jako mnożenie dzielnej przez odwrotność dzielnika. W projekcie *Aukciszek* odwrotność modulo p oblicza się przy użyciu rozszerzonego algorytmu Euklidesa, którego złożoność to $O(\log(p))$.

Bezpieczeństwo schematu Shamira

Niech s to ustalony sekret z $\mathbb{Z}_p$ i niech $x_1, \ldots, x_t$ to niezerowe i parami różne punkty z $\mathbb{Z}_p$. Istnieje bijekcja $(a_1, \ldots, a_{t-1}) \in \mathbb{Z}_p^{t-1} \to (y_1, \ldots, y_{t-1}) \in \mathbb{Z}_p^{t-1}$, gdzie $y_i = f(x_i)$ dla $i = 1, \ldots, t-1$ oraz $f(x) = s + a_1 x + \ldots + a_{t-1} x^{t-1}$.

Dowód. Każdy wielomian przyjmuje dokładnie jedną wartość w punkcie x_i, dlatego nie istnieją dwa ciągi $(y_1, \ldots, y_{t-1})$, $(y'_1, \ldots, y'_{t-1})$ odpowiadające temu samemu $(a_1, \ldots, a_{t-1})$. Jeżeli istniałyby dwa ciągi $(a_1, \ldots, a_{t-1})$, $(a'_1, \ldots, a'_{t-1})$ odpowiadające temu samemu $(y_1, \ldots, y_{t-1})$, oznaczałoby to, że wielomiany $g(x) = s + a_1 x + \ldots + a_{t-1} x^{t-1}$ i $h(x) = s + a'_1 x + \ldots + a'_{t-1} x^{t-1}$ przyjmowałyby takie same wartości w t różnych punktach: $t-1$ punktach x_i i w punkcie 0. Wtedy $g(x)$ i $h(x)$ byłyby tym samym wielomianem. Stąd istnieje iniekcja $(a_1, \ldots, a_{t-1}) \in \mathbb{Z}_p^{t-1} \to (y_1, \ldots, y_{t-1}) \in \mathbb{Z}_p^{t-1}$. Ponieważ jej dziedzina i przeciwdziedzina to ten sam skończony zbiór, jest ona również bijekcją. □

[1] Interpolacja o złożoności $O(n^2)$ jest oceniana w pracy [Sha79] jako wystarczająco efektywna w zastosowaniach praktycznych, choć autor zaznacza, że istnieją sposoby osiągnięcia złożoności $O(n \cdot \log^2(n))$.

Istnienie opisanej bijekcji implikuje, że kiedy a_i to zmienne losowe z $\mathbb{Z}_p$ o rozkładzie jednostajnym, wtedy zmienne y_i mają taki sam rozkład. Skoro rozkład wartości zbioru $t-1$ udziałów jest jednostajny dla każdego możliwego s, schemat podziału sekretu Shamira jest bezpieczny [BS23, (22.1.2)].

Na podstawie zbioru $t-1$ udziałów nie można uzyskać żadnej informacji o wartości sekretu. Tę własność można zobrazować za pomocą gry.

Przykład 2.7. Gra prowadzona między stroną dzielącą sekret C i nieograniczonym obliczeniowo adwersarzem $\mathcal{A}$, w grupie skończonej $\mathbb{Z}_p$, przy publicznie znanych parametrach p, n, t i zbiorze punktów $x_1, \ldots, x_n$, $x_i \in \mathbb{Z}_p$.

1. C dokonuje podziału sekretu $s \in \mathbb{Z}_p$ pomiędzy n stron z progiem rekonstrukcji równym t, otrzymując zbiór n udziałów: $[s] = (x_i, y_i) : i = 1, \ldots, n$.
2. C losuje podzbiór udziałów $A \subset [s]$, $|A| = t-1$ i wysyła go $\mathcal{A}$.
3. $\mathcal{A}$ wybiera $s' \in Z$.

$\mathcal{A}$ wygrywa, jeżeli $s' = s$.

Skoro rozkład wartości zbioru $t-1$ udziałów jest jednostajny, otrzymany podzbiór A mógł powstać na podstawie dowolnej wartości s. Niech x_j będzie ustalonym punktem spośród $x_1, \ldots, x_n$, który nie występuje w A. Wtedy dla każdego $y' \in \mathbb{Z}_p$ zbiór A z dodanym udziałem postaci (x_j, y') może posłużyć do rekonstrukcji innego s'. Jeżeli $A \cup \{(x_j, y'_1)\}$ oraz $A \cup \{(x_j, y'_2)\}$ wyznaczają to samo s', wtedy y'_1 i y'_2 są sobie równe. Jest tak dlatego, ponieważ $A \cup \{(0, s')\}$ jednoznacznie wyznacza wielomian, na którym musi leżeć zarówno (x_j, y'_1), jak i (x_j, y'_2). Stąd liczba możliwych wartości sekretu, na podstawie którego wygenerowano A, wynosi $|\mathbb{Z}_p| = p$.

Prawdopodobieństwo wygranej $\mathcal{A}$ nie jest istotnie większe od prawdopodobieństwa wygranej przy losowym wyborze s'. Dla każdego możliwego $\mathcal{A}$, n, t zachodzi:

$$\mathbb{P}(s' = s) = \frac{1}{p} + \epsilon,$$

gdzie ϵ jest pomijalnie małą wartością.

Podsumowanie własności schematu podziału sekretu Shamira na podstawie [VMV96, (12.72)]

— Znajomość $t-1$ udziałów nie daje żadnej wiedzy o sekrecie. Dla każdego zbioru udziałów $\{s_1, \ldots, s_{t-1}\}$ istnieje $p-1$ różnych możliwych wartości s_t, dających p różnych możliwych wartości s, z których każda jest równie prawdopodobna.

— Rozmiar udziału jest równy rozmiarowi sekretu - jednej liczbie z $\mathbb{F}_p$. Używanie udziałów większych niż sekret jest nieefektywne. Używanie udziałów

mniejszych niż sekret zmniejsza potencjalne bezpieczeństwo. Używając ataku brutalnego łatwiej wtedy zgadnąć wartość udziału, niż sekretu.

— Istnieje możliwość generowania nowych udziałów bez unieważniania istniejących. Można przez to zwiększyć liczbę udziałowców, bez zmiany liczby stron potrzebnych do rekonstrukcji sekretu.

— Strony, pomiędzy które dzielony jest sekret, mogą mieć różniącą się liczbę udziałów, a przez to mniejszy lub większy wpływ na rekonstrukcję sekretu. Np. prezydent z przykładu 2.1 mógłby otrzymać kilka udziałów (punktów na wielomianie), zatem potrzebowałby współpracy mniejszej liczby osób, żeby poznać sekret.

— Bezpieczeństwo schematu nie opiera się na nieudowodnionych własnościach, takich jak nierówność klas problemów P i NP.

2.2. Operacje na podzielonych sekretach

Podział sekretu może być wykorzystywany nie tylko do przechowywania informacji. Pomimo utajnienia wartości przez rozbicie ich na udziały, możliwe jest wykonywanie operacji arytmetycznych na sekretach. W tym celu będziemy używać metod opisanych w pracy wprowadzającej protokół mnożenia sekretów BGW (Ben-Or - Goldwasser - Wigderson) [WOG88].

Zakładamy, że n stron posiada po jednym udziale, tak że udziałowiec P_i dysponuje udziałem s_i. Znajomość t udziałów s_i umożliwia rekonstrukcję sekretu s. Strony mogą modyfikować swoje udziały oraz komunikować się z innymi stronami. Wtedy, wykonując odpowiedni program T złożony z tych operacji, udziałowcy mogą przekształcić swoje udziały $s_i \xrightarrow{T} s'_i$. Użyte do rekonstrukcji, s'_i wyznaczą sekret s'. Przy poprawnych założeniach, konstruując odpowiedni program T, można osiągnąć przekształcenie udziałów odpowiadające dowolnej funkcji $F(s) = s', F : \mathbb{F}_p \to \mathbb{F}_p$.

Stosunek wartości n i t ustala się w zależności od pożądanych właściwości protokołu oraz przewidywanego zachowania stron. W pracy [WOG88], autorzy określają właściwość zachowania prywatności (*t-privacy*) oraz zachowania poprawności (*t-resilience*)[2]. Protokół zachowuje prywatność, jeżeli dowolna grupa licząca mniej niż t stron w czasie wykonywania działań nie dowiaduje się o sekrecie nic więcej ponad to, co mogła wywnioskować ze zbioru swoich udziałów. Protokół zachowuje poprawność, jeżeli mniej niż t stron, złośliwie modyfikując wysyłane przez siebie komunikaty w trakcie obliczeń, nie może

[2] W pracy [WOG88] używane jest oznaczenie t, które odpowiada oznaczeniu $k-1$ z pracy [Sha79]. W *Aukciszku* przyjmujemy oznaczenie t, odpowiadające k w [Sha79] oraz $t+1$ w [WOG88].

zmienić wyniku rekonstrukcji, jeżeli będzie do niej użyty każdy z n udziałów. Autorzy udowadniają, że własność prywatności zachowują zawsze tylko te protokoły, które spełniają warunek $t-1 < n/2$, a własność poprawności - warunek $t-1 < n/3$.

Używając właściwości kodów korekcyjnych, w przypadku $t-1 < n/3$, można zapewnić, że nawet jeżeli $t-1$ stron nie dostarczy swoich udziałów albo złośliwie je zmieni, używając wszystkich n udziałów do rekonstrukcji, można poprawić $t-1$ błędów i poprawnie zrekonstruować wynik działania [WOG88].

Kiedy co najmniej t stron nie jest niezależnych, ale jest w zmowie i dzieli się ze sobą swoimi udziałami, schemat podziału nie ma sensu, bo odpowiada sytuacji, w której sekret byłby ujawniony grupie stron będącej w zmowie, a podzielony tylko pomiędzy pozostałe strony. Dlatego przy użyciu schematu podziału sekretu zakłada się, że nie występują takie zmowy. W przypadku kiedy $t-1 < n/2$, mówi się o zachowaniu zaufanej większości. Oznacza to, że strony, pomiędzy które dzielony jest sekret, są na tyle niezależne od siebie, że nie występuje wśród nich zmowa obejmująca większości z nich.

W systemie *Aukciszek* zakładamy, że upoważnione do przeprowadzania aukcji strony są tak dobrane, że nie będą dążyły do obstrukcji aukcji. Dlatego rezygnujemy z warunku zachowania poprawności i wymogu $t-1 < n/3$. Ważne jest dla nas zachowanie prywatności, dlatego ustalamy odpowiedni próg rekonstrukcji sekretu $t-1 < n/2$ i zakładamy zachowanie zaufanej większości[3]. Zakładamy, że strony objęte zmową mogą jedynie wymieniać się swoimi udziałami i komunikatami, które otrzymują od stron nieobjętych zmową w trakcie obliczeń. Złośliwe strony mogą osiągnąć jedynie uniemożliwienie wykonania obliczeń, nie poznają jednak sekretnych wartości.

Złożoność obliczenia funkcji F z zachowaniem prywatności jest ograniczona z góry przez złożoność F przemnożoną przez czynnik wielomianowy zależny od n [WOG88].

2.2.1. Mnożenie wielomianu przez stałą

Opiszemy teraz operację mnożenia podzielonego sekretu przez pewną jawną stałą wartość, np. -1. Mnożenie wielomianu przez stałą nie może podnieść jego stopnia. Zauważmy, że jeżeli

$$k \in \mathbb{F}_p,\ w(x) \in \mathbb{F}_p[X],\ deg(w(x)) < t,$$

to:

$$k \cdot w(x) = k \cdot (w_0 + w_1 x + \ldots + w_{t-1} x^{t-1}) = k \cdot w_0 + k \cdot w_1 x + \ldots + k \cdot w_{t-1} x^{t-1}.$$

[3] Obecnie w *Aukciszku* $t = \lfloor \frac{n-1}{2} \rfloor$, chociaż bez naruszania warunku można przyjąć $t = \lfloor \frac{n+1}{2} \rfloor$

Jeżeli w punkcie x wartość wielomianu $w(x)$ wynosiła y, wartość wielomianu $k \cdot w(x)$ w tym samym punkcie będzie wynosić $k \cdot y$. Dlatego strony, po przekształceniu swoich udziałów $(x, y) \xrightarrow{T} (x, k \cdot y)$, mogą dokonać rekonstrukcji wielomianu $k \cdot w(x)$.

Protokół 2.2. Mnożenie sekretu podzielonego schematem Shamira przez stałą.
Dane:
— $t - 1 < n/2$,
— $[s] = \{s_1, \ldots, s_n\}, s_i \in \mathbb{F}_p$,
— $k \in \mathbb{F}_p$.
gdzie:
— $J \subseteq \{1, \ldots, n\}, |J| = t$,
— $C(J, \{s_j\}_{j \in J}) \to s$.
Przebieg:
1. Każda ze stron P_i niezależnie oblicza nowy udział $s'_i = s_i \cdot k \ (mod\ p)$.
Wyjście:
— $[s'] = \{s'_1, \ldots, s'_n\}, s'_i \in \mathbb{F}_p$.
gdzie:
— $J \subseteq \{1, \ldots, n\}, |J| = t$,
— $C(J, \{s'_j\}_{j \in J}) \to s' = s \cdot k \ (mod\ p)$.

Jeżeli współczynniki wielomianu, który był użyty do podzielenia sekretu s, były nierozróżnialne od losowych, współczynniki wielomianu ukrywającego s' również są nierozróżnialne od losowych [WOG88].

Złożoność obliczeniowa mnożenia przez stałą

Każda z n stron wykonuje jedno mnożenie udziału przez stałą. Nie jest wymagana komunikacja między stronami. Złożoność mnożenia sekretu przez stałą wynosi $O(n)$.

2.2.2. Dodawanie wielomianów

W tej części opiszemy, jak uzyskać podział sekretu, ukrywający sumę dwóch podzielonych wcześniej sekretów. Suma wielomianów nie może być wielomianem wyższego stopnia niż wielomian o najwyższym stopniu spośród czynników dodawania. Załóżmy, że:

$$w(x), g(x) \in \mathbb{F}_p[X],\ deg(w(x)) \leqslant deg(g(x)) < t.$$

Wtedy:

$$w(x) + g(x) = (w_0 + w_1 x + \ldots + w_{t-1} x^{t-1}) + (g_0 + g_1 x + \ldots + g_{t-1} x^{t-1})$$

$$w(x) + g(x) = (w_0 + g_0) + (w_1 + g_1)x + \ldots + k \cdot (w_{t-1} + g_{t-1})x^{t-1}.$$

Jeżeli w punkcie x wartość wielomianu $w(x)$ wynosiła y_w, a wartość wielomianu $g(x)$ wynosiła y_g, wtedy wartość wielomianu $w(x) + g(x)$ w tym samym punkcie będzie wynosić $y_w + y_g$. Dlatego strony, po obliczeniu nowych udziałów $(x, y_w), (x, y_g) \xrightarrow{T} (x, y_w + y_g)$, mogą dokonać rekonstrukcji wielomianu $w(x) + g(x)$.

Protokół 2.3. Dodawanie sekretów podzielonych schematem Shamira.
Dane:
— $t - 1 < n/2$,
— $[a] = \{a_1, \ldots, a_n\}, a_i \in \mathbb{F}_p$,
— $[b] = \{b_1, \ldots, b_n\}, b_i \in \mathbb{F}_p$.
gdzie:
— $J \subseteq \{1, \ldots, n\}, |J| = t$,
— $C(J, \{a_j\}_{j \in J}) \to a$,
— $C(J, \{b_j\}_{j \in J}) \to b$.
Przebieg:
1. Każda ze stron P_i niezależnie oblicza nowy udział $c_i = a_i + b_i \ (mod\ p)$.
Wyjście:
— $[c] = \{c_1, \ldots, c_n\}, c_i \in \mathbb{F}_p$.
gdzie:
— $J \subseteq \{1, \ldots, n\}, |J| = t$,
— $C(J, \{c'_j\}_{j \in J}) \to c = a + b \ (mod\ p)$.

Jeżeli współczynniki co najmniej jednego z wielomianów, które były użyte do podzielenia sekretów a, b, były nierozróżnialne od losowych, współczynniki wielomianu ukrywającego c również są nierozróżnialne od losowych [WOG88].

Złożoność obliczeniowa dodawania

Każda z n stron wykonuje jedno dodawanie udziałów. Nie jest wymagana komunikacja między stronami. Złożoność dodawania sekretów wynosi zatem $O(n)$.

2.2.3. Wykrycie i naprawa błędów rekonstrukcji sekretu

Nieuczciwe strony mogą podać udziały inne niż otrzymane w wyniku poprawnego wykonania działania. Jeżeli działanie nie zmienia stopnia wielomianu, wykrycie i naprawienie błędu zachodzi identycznie jak w zwykłym przypadku schematu podziały Shamira. Nawet jeśli niektóre strony podały błędne udziały do rekonstrukcji, można skorzystać z poprawnych udziałów, o ile to one przeważają, i zarówno wyznaczyć poprawny sekret, jak i rozpoznać błędne

udziały. W tym celu używa się wykazywanych przez schemat podziału Shamira właściwości kodów korekcyjnych.

Kody korekcyjne

Schemat Shamira z odpowiednimi wartościami n i t realizuje konstrukcję kodów korekcyjnych Reeda-Solomona i Reed–Mullera. Następujący opis kodowania i naprawy błędów bazuje na książce [Sma16, (19.3.)].

Kody korekcyjne opierają się na mapowaniu danych na słowa kodowe. Aby skorzystać z kodowania korekcyjnego Reeda-Solomona, należy przedstawić dane w postaci wielomianów stopnia k z $\mathbb{F}_p[X]$. Słowa kodowe to wartości wielomianu w danych n punktach. Stąd do zakodowania $k+1$ liczb z $\mathbb{F}_p$ ($(k+1) \cdot \lfloor \log_2 p \rfloor$ bitów danych) - jednego wielomianu stopnia k z $\mathbb{F}_p[X]$, danego współczynnikami $a_0, \cdots, a_k$ - potrzeba słowa kodowego będącego wektorem wartości wielomianu w danych punktach - n liczb z $\mathbb{F}_p$ ($n \cdot \lfloor \log_2 p \rfloor$ bitów danych).

Słowa kodowe to wektory z $\mathbb{F}_p$, gdzie p jest liczbą pierwszą. Używając kodu o długości słowa równej n, można poprawić $\lfloor \frac{n-k-1}{2} \rfloor$ błędów.

Definicja 2.2. Kody korekcyjne Reeda-Solomona [Sma16, (19.3.)].
Parametry:
- p - liczba pierwsza,
- $k+1$ - długość danych,
- $\mathbb{D} = \{a_0 + a_1 x + \cdots + a_k x^k : a_i \in \mathbb{F}_p\}$ - zbiór wielomianów stopnia $\leqslant k$,
- n - długość słowa kodowego,
- $X \subset \mathbb{F}_p, |X| = n$ - zbiór współrzędnych,
- $\mathbb{C} = \{(f(x_1), \cdots, f(x_n)) : f \in \mathbb{D}\}$ - zbiór słów kodowych,
- $e = \lfloor \frac{n-k-1}{2} \rfloor$ - liczba błędów możliwych do naprawy.

Kodowanie:
1. Przedstawienie danych jako $k+1$ liczb z $\mathbb{F}_p$ - współczynników wielomianu $f(x)$.
2. Obliczenie słowa kodowego $c = (f(x_1), \cdots, f(x_n)),\ x_i \in X$.

Dekodowanie:
1. Interpretacja słowa kodowego $c \in \mathbb{C}$ jako n punktów $(x, y) :\ x \in X,\ y \in \mathbb{F}_p$.
2. Znalezienie pozycji e błędnych punktów i odrzucenie ich.
3. Wyznaczenie wielomianu $f(x)$ na podstawie $n - e$ poprawnych punktów.
4. Odczytanie danych przedstawionych jako współczynniki wielomianu $f(x)$.

W przypadku schematu podziału sekretu Shamira używa się zbioru $\mathbb{D} = \{a_0 + a_1 x + \cdots + a_k x^k : a_i \in \mathbb{F}_p\}$ wielomianów stopnia k, otrzymując p^{k+1} możliwych wielomianów, ukrywających każdy z sekretów $0, \cdots, p-1$ na p^k sposobów.

Zbiór X to zbiór współrzędnych x udziałów, $X = \{x_1, \cdots, x_n\}$. Parametr t schematu Shamira, określający liczbę udziałów potrzebną do rekonstrukcji sekretu, wyznacza parametr $k = t - 1$.

Zakładając, że wśród n punktów występuje co najwyżej e błędów, można je poprawić, sprawdzając wszystkie możliwe rekonstrukcje używające podzbiorów X mocy $n - e$. Jeżeli podzbiór wyznacza wielomian stopnia większego niż k, na pewno zawiera błędny punkt. Jeżeli wyznacza wielomian stopnia k, prawdopodobnie zawiera on jedynie poprawne punkty, choć zależy to od stosunku $n - e$ i k. Oczywiście $n - e$ musi być większe od k, ale kiedy $n - e = k - 1$, każdy podzbiór długości $n - e$ wyznacza wielomian stopnia k. Im większa różnica między $n - e$ a k, tym mniejsze prawdopodobieństwo, że zrekonstruowany będzie poprawny wielomian stopnia k.

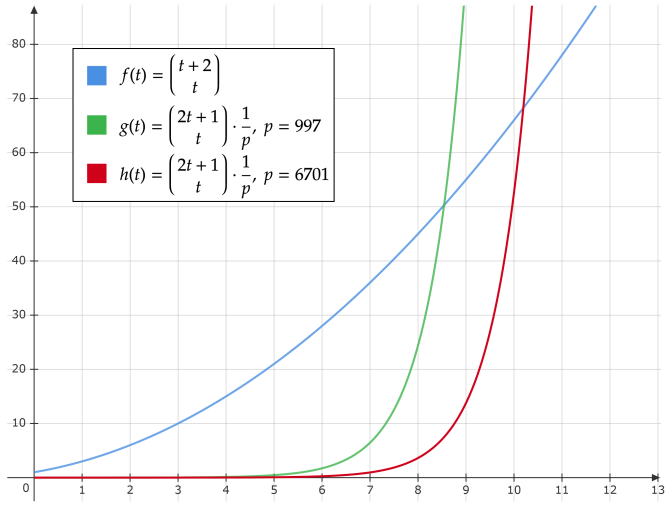
Przykład - prawdopodobieństwo wykrycia błędu w przypadku $n = 2t + 1$

W przypadku $n = 2t + 1$, jeżeli do rekonstrukcji zostanie użyte $t - 1$ błędnych udziałów s'_i i jeden poprawny udział s_i, rekonstrukcja zwróci pewną liczbę $s' \neq s$. Nawet jeśli nieuczciwe strony współpracowały ze sobą, nie mają jednak pełnej kontroli nad s'. Wartość s' zależy od poprawnego udziału, który może dowolnie wpłynąć na rekonstruowany wielomian i dać losową wartość sekretu. Jeżeli użyte zostanie co najwyżej $t - 1$ błędnych udziałów i co najwyżej t poprawnych, możliwe, że rekonstrukcja powiedzie się i zrekonstruowany zostanie błędny sekret, a strony nie dowiedzą się o tym. Jeżeli do rekonstrukcji użyte zostaną wszystkie udziały: $t - 1$ błędne i $t + 2$ poprawne, nie mogą wyznaczyć innego wielomianu stopnia $t - 1$ poza poprawnym.

Kiedy wykorzystuje się wszystkie udziały, rekonstrukcja na pewno nie zwróci wielomianu stopnia t. Dowiedziawszy się w ten sposób, że w zbiorze udziałów są błędy, strony mogą przeanalizować wszystkie możliwe rekonstrukcje wielomianu stopnia $t - 1$ na podstawie podzbioru t udziałów. Poprawny wynik s zostanie obliczony co najmniej $\binom{t+2}{t}$ razy, czyli zawsze wtedy, kiedy zostaną wybrane jedynie poprawne udziały. Będzie to najpewniej najczęstsza wartość rekonstrukcji, pozostałe powinny być równomiernie rozłożone, niezależnie od tego, czy są wyznaczane przez zbiory $t - 1$ błędnych i 1 poprawny, $t - 1$ poprawnych i 1 błędnego, lub dowolne inne kombinacje. Każda wartość $s' \neq s$ będzie otrzymywana z prawdopodobieństwem $\frac{1}{|\mathbb{F}_p|}$, czyli oczekiwana liczba jej wystąpień będzie wynosić $\binom{2t+1}{t} \cdot \frac{1}{|\mathbb{F}_p|}$. To, czy s będzie najczęstszą wartością rekonstrukcji, zależy od liczby $|\mathbb{F}_p| = p$.

Oczekiwana liczba wystąpień s to $\binom{t+2}{t} + (\binom{2t+1}{t} - \binom{t+2}{t}) \cdot \frac{1}{p}$. Oczekiwana liczba wystąpień ustalonego $s' \neq s$ to $(\binom{2t+1}{t} - \binom{t+2}{t}) \cdot \frac{1}{p}$.

Jeżeli wykorzystywane wartości t są niewielkie, liczba kombinacji nie jest



Rysunek 2.1. Wartość oczekiwana liczby rekonstrukcji s' w zależności od t i p.

przytłaczająca. Biorąc pod uwagę niewielkie koszty obliczeń, możliwe jest sprawdzenie wszystkich możliwości w celu wyłonienia grupy niepoprawnych udziałów. Niemniej jest to zupełnie nieefektywne, bo opiera się na prawdopodobieństwie i nie jest uniwersalne, nie sprawdzi się już np. kiedy $n = 2t - 1$. A w omawianym przypadku $n = 2t + 1$, jeżeli jednocześnie $t < 4$, spełniony jest warunek $t - 1 < n/3$. Wtedy można zastosować właściwości kodów korekcyjnych do naprawienia $t - 1$ błędów i odczytania poprawnego wyniku dodawania.

Naprawa $n/3$ błędów

Kodowanie Reeda-Solomona zwraca n punktów wielomianu stopnia k. Do dekodowania używa się $n - s$ z nich, gdzie s to liczba brakujących punktów - utraconych w wyniku awarii komunikacji lub nieprzesłanych przez strony. Prawidłowe dekodowanie polega na jednoznacznym wyznaczeniu wielomianu stopnia k, przechodzącego przez $n - s - e$ ze zbioru $n - s$ punktów, gdzie e to liczba błędnych punktów. Dekodowanie powiedzie się zawsze, kiedy dysponuje się co najmniej $k + 2e$ punktami. Stąd zakłada się $n > k + 2e + s$ [Sma16, (19.3.3.)].

W schemacie podziału sekretu Shamira można zapewnić możliwość naprawy $t - 1$ błędów. Jest to maksymalna wielkość podzbioru udziałów niepozwalającą na rekonstrukcję - dopuszczalna liczba nieuczciwych stron. Jako zagrożenie rozpatrywane są jedynie błędy, a nie utrata danych, dlatego przyjmuje się $s = 0$. Ustalając $k = t - 1$, $s = 0$, $e = t - 1$ i podstawiając w zależności $n > k + 2e + s$,

otrzymuje się:

$$n > t - 1 + 2(t - 1)$$

$$n > 3(t - 1)$$

$$t - 1 < n/3.$$

Jest to jednoznaczne z warunkiem zachowania poprawności z pracy [WOG88].

Algorytm Berlekampa-Welcha [Sma16, (19.3.4.)]

Algorytm Berlekampa-Welcha w efektywny obliczeniowo sposób dekoduje kody korekcyjne Reeda-Solomona w przypadku $k < n/3$. Pozwala na wyznaczenie poprawnego wielomianu ukrywającego dane oraz współrzędnych błędnych punktów.

Wejściem do algorytmu jest zbiór punktów $\{(x_i, y_i) : i = 1, \ldots, n\}$. Zakładamy, że co najwyżej k z nich jest błędnych. Wyjście to wzór wielomianu $a(x)$ stopnia k, ukrywającego sekret s, oraz współrzędne błędnych punktów, dla których $y_i \neq a(x_i)$.

Ustalmy funkcję dwóch zmiennych $Q(x, y) = d(x) - c(x) \cdot y$, tak aby przyjmowała wartość 0 dla każdego z punktów (x_i, y_i). Istnieje para wielomianów: $c(x)$ stopnia k i $d(x)$ stopnia co najwyżej $2k$, która spełnia zależność $Q(x_i, y_i) = d(x_i) - c(x_i) \cdot y_i = 0$, dla $i = 1, \ldots, n$. Wielomian $d(x)$ jest równy iloczynowi $a(x)$ i $c(x)$. W poprawnych punktach wartość y_i jest równa $a(x_i)$. Wartość $Q(x_i, y_i)$ jest równa 0, ponieważ $d(x_i) = c(x_i) \cdot a(x_i)$.

$$Q(x_i, y_i) = d(x_i) - c(x_i) \cdot y_i$$

$$Q(x_i, y_i) = d(x_i) - c(x_i) \cdot a(x_i)$$

$$Q(x_i, y_i) = c(x_i) \cdot a(x_i) - c(x_i) \cdot a(x_i) = 0$$

W błędnych punktach wartość y_i nie jest równa $a(x_i)$. Aby $Q(x_i, y_i)$ było równe 0, wartość $c(x_i)$ musi wynosić 0.

$$Q(x_i, y_i) = d(x_i) - c(x_i) \cdot y_i$$

$$Q(x_i, y_i) = c(x_i) \cdot a(x_i) - c(x_i) \cdot y_i$$

$$Q(x_i, y_i) = 0 \cdot a(x_i) - 0 \cdot y_i = 0$$

Rozwiązując układ równań $Q(x_i, y_i) = 0$ dla $i = 1, \ldots, n$ można wyznaczyć współczynniki wielomianów $c(x)$ i $d(x)$. Na ich podstawie można ujawniać wielomian użyty do kodowania $a(x) = \frac{d(x)}{c(x)}$. Pozwala to również określić współrzędne błędnych punktów, czyli tych, dla których $a(x_i) \neq y_i$. Będą to miejsca zerowe wielomianu $c(x)$.

Wielomian $d(x)$ stopnia $2k$ ma $2k+1$ nieznanych współczynników d_i, a $c(x)$ stopnia k ma $k+1$ nieznanych współczynników c_i. Dysponując $n = 3k+1$ punktami, nie można rozwiązać układu z $3k+2$ niewiadomymi. Należy zmniejszyć o jeden liczbę zmiennych do wyznaczenia. Skorzystamy z faktu, że punkt o współrzędnej x równej 0 z definicji nie należy do udziałów, ponieważ ukrywa on sekret, tzn. $a(0) = s$. Wymagamy spełnienia równości $d(0) = c(0) \cdot s$. Bez wpływu na korekcję błędów w punktach (x_i, y_i) możemy ustalić $c(0) = c_0 = 1$, ograniczając liczbę zmiennych do wyznaczenia do $3k+1$.

Na podstawie $n > 3k$ punktów (x_i, y_i) otrzymujemy układ równań z $3k+1$ niewiadomymi.

$$Q(x_i, y_i) = 0$$

$$d(x_i) - c(x_i) \cdot y_i = 0$$

$$(d_0 + d_1 x_i + \ldots + d_{2k} x_i^{2k}) - (1 + c_1 x_i + \ldots + c_k x_i^k) \cdot y_i = 0$$

$$(d_0 + d_1 x_i + \ldots + d_{2k} x_i^{2k}) - (y_i + c_1 x_i y_i + \ldots + c_k x_i^k y_i) = 0$$

$$d_0 + d_1 x_i + \ldots + d_{2k} x_i^{2k} - c_1 x_i y_i - \ldots - c_k x_i^k y_i = y_i$$

Przykład 2.8. Naprawa błędów rekonstrukcji schematu Shamira za pomocą algorytmu Berklampa-Welcha.

Dane:

— $n = 6$, $t = 2$,

— $p = 23$,

— $s = 17$.

Podział sekretu:

1. $a_0 = s = 17 = -6$
2. $a_1 = 3 \xleftarrow{R} \mathbb{F}_{23}$
3. $x_1, \ldots, x_5 = 1, \ldots, 6$
4. $f(1) = 20 = -3, f(2) = 0, f(3) = 3, f(4) = 6, f(5) = 9, f(6) = 12$

Otrzymane do rekonstrukcji punkty:

— $(1, -3), (2, 0), (3, 3), (4, 12), (5, 9), (6, 12)$.

Konstrukcja układu równań

$$\begin{bmatrix} 1 & 1^2 & 1^3 & -1^1 \cdot y_1 \\ 1 & 2^2 & 2^3 & -2^1 \cdot y_2 \\ 1 & 3^2 & 3^3 & -3^1 \cdot y_3 \\ 1 & 4^2 & 4^3 & -4^1 \cdot y_4 \\ 1 & 5^2 & 5^3 & -5^1 \cdot y_5 \\ 1 & 6^2 & 6^3 & -6^1 \cdot y_6 \end{bmatrix} \cdot \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ c_1 \end{bmatrix} \equiv \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix} \pmod{23}$$

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 1 & 2 & 4 & 0 \\ 1 & 3 & 9 & -9 \\ 1 & 4 & 16 & -48 \\ 1 & 5 & 25 & -45 \\ 1 & 6 & 36 & -72 \end{bmatrix} \cdot \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ c_1 \end{bmatrix} \equiv \begin{bmatrix} -3 \\ 0 \\ 3 \\ 12 \\ 9 \\ 12 \end{bmatrix} \ (mod\ 23)$$

$$\begin{bmatrix} 1 & 1 & 1 & 3 \\ 1 & 2 & 4 & 0 \\ 1 & 3 & 9 & 14 \\ 1 & 4 & 16 & 21 \\ 1 & 5 & 2 & 1 \\ 1 & 6 & 13 & 3 \end{bmatrix} \cdot \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ c_1 \end{bmatrix} \equiv \begin{bmatrix} 20 \\ 0 \\ 3 \\ 12 \\ 9 \\ 12 \end{bmatrix} \ (mod\ 23)$$

Rozwiązanie wynosi zatem:

$$c(x) = 1 - \frac{1}{4}x \equiv 1 + 17x \ (mod\ 23),$$

$$d(x) = -6 - \frac{9}{2}x - \frac{3}{4}x^2 \equiv 17 + 16x + 5x^2 \ (mod\ 23).$$

Można teraz wyznaczyć wzór prawidłowego wielomianu kodującego sekret:

$$a(x) = \frac{d(x)}{c(x)} = \frac{17 + 16x + 5x^2}{1 + 17x} = 17 + 3x \equiv -6 + 3x \ (mod\ 23).$$

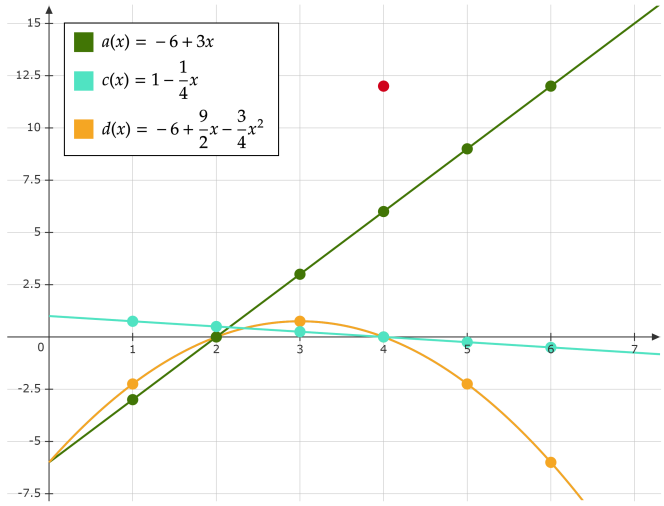
Lokalizacja błędnego punktu to:

$$(x_4, y_4) : \ y_4 - a(x_4) \neq 0$$

$$(4, 12) : \ 12 - a(4) = 12 - 6 = 6 \neq 0.$$

Używając n punktów, otrzymuje się układ n równań. Złożoność obliczeniowa algorytmu Berlekampa-Welcha sprowadza się do złożoności rozwiązania układu, tzn. $O(n^3)$.

Algorytm Berlekampa-Welcha to jeden z najstarszych sposobów dekodowania kodów Reeda-Solomona. Jest efektywny w przypadku $k < n/3$, ale nie zadziała, jeżeli $k \geqslant n/3$, nawet jeśli jednocześnie $k < n/2$, $k + 2e + s < n$ i $e = \lfloor \frac{n-k-1}{2} \rfloor$ (np. $k = 2, e = 1, s = 0, n = 5$).



Rysunek 2.2. Przykładowe wielomiany używane w algorytmie Berklampa-Welcha. Dla przejrzystości wykresu pominięto redukcję modulo. Kolorem czerwonym zaznaczony jest błędny punkt.

2.3. Mnożenie sekretów

W tym podrozdziale zostanie omówiony protokół mnożenia sekretów BGW (Ben-Or - Goldwasser - Wigderson) na podstawie wprowadzającej go pracy [WOG88].

Naszym celem jest otrzymanie nowego podziału sekretu ukrywającego iloczyn dwóch podzielonych wcześniej sekretów. Iloczyn wielomianów może być wielomianem wyższego stopnia niż wielomian o najwyższym stopniu spośród czynników mnożenia. Dlatego rekonstrukcja iloczynu nie jest możliwa dla dowolnych wartości $t \leqslant n$. Natomiast wielokrotne mnożenie wymusza redukcję stopnia wielomianu, która odbywa się dzięki komunikacji stron.

Niech $w(x), g(x)$ to wielomiany ukrywające sekrety a, b.

$$w(x), g(x) \in \mathbb{F}_p[X],\ deg(w(x)) \leqslant deg(g(x)) < t$$

Wtedy:

$$w(x) \cdot g(x) = (w_0 + w_1 x + \ldots + w_{t-1}x^{t-1}) \cdot (g_0 + g_1 x + \ldots + g_{t-1}x^{t-1})$$

$$w(x) \cdot g(x) = (w_0 g_0 + \ldots + w_0 g_{t-1}x^{t-1}) + \ldots + (w_{t-1}g_0 x^{t-1} + \ldots + w_{t-1}g_{t-1}x^{2(t-1)})$$

$$w(x) \cdot g(x) = (w_0 g_0) + (w_0 g_1 + w_1 g_0)x + \ldots + (w_{t-1}g_{t-1})x^{2(t-1)}.$$

Jeżeli w punkcie x wartość wielomianu $w(x)$ wynosiła y_w, a wartość wielomianu $g(x)$ wynosiła y_g, wtedy wartość wielomianu $w(x) \cdot g(x)$ w tym samym punkcie będzie wynosić $y_w \cdot y_g$. Strony, po obliczeniu nowych udziałów $(x, y_w), (x, y_g) \xrightarrow{T} (x, y_w \cdot y_g)$, mogą dokonać rekonstrukcji wielomianu $w(x) \cdot g(x)$ oznaczanego dalej $h(x)$, o ile posiadają wystarczającą liczbę udziałów.

Strona P_i, po pomnożeniu swoich udziałów a_i, b_i, otrzymuje udział iloczynu. Wynikiem mnożenia dwóch wielomianów $w(x), g(x)$ stopnia $t-1$ jest wielomian $h(x)$ stopnia $2(t-1)$. Żeby wyznaczyć jego wzór, potrzeba znajomości co najmniej $2t-1$ należących do niego punktów. Aby wyznaczenie iloczynu było możliwe, liczba stron protokołu musi wynosić co najmniej $2t-1$, co jest jednoznaczne z warunkiem zachowania prywatności $t-1 < n/2$.

Spełnienie tej zależności pozwala na wykonanie jednego mnożenia, jednak każde kolejne ponownie podwaja stopień wielomianu. Żeby umożliwić dowolną liczbę iloczynów, po każdym mnożeniu konieczne jest sprowadzenie wyniku do wielomianu stopnia $t-1$. Okazuje się bowiem, że na podstawie n punktów wielomianu $h(x)$ stopnia $2(t-1)$ można wyznaczyć n punktów wielomianu $k(x)$ stopnia $t-1$, który będzie ukrywał ten sam sekret, tzn. $k_0 = h_0$.

2.3.1. Redukcja stopnia wielomianu

Opisana poniżej metoda redukcji stopnia wielomianu ukrywającego sekret, polegająca na wyznaczeniu wielomianu $k(x)$ stopnia $t-1$ na podstawie wielomianu $h(x)$ stopnia $2(t-1)$, pochodzi z pracy [WOG88].

Niech wielomian ukrywający iloczyn sekretów ma postać

$$h(x) = h_0 + h_1 x + \ldots + h_{2(t-1)} x^{2(t-1)}.$$

Wtedy:

$$s_i = h(x_i) = w(x_i) g(x_i),$$

gdzie s_i to udział strony P_i, dla $i = 1, \ldots, n$. Redukcja stopnia będzie obliczeniem wielomianu $k(x)$ stopnia $t-1$, takiego że

$$k(x) = h_0 + h_1 x + \ldots + h_{t-1} x^{t-1},$$

i jest równoważna ze znalezieniem przez stronę P_i wartości

$$r_i = k(x_i),$$

dla $i = 1, \ldots, n$.
Niech

$$S = (s_1, \ldots, s_n),$$

$$R = (r_i, \ldots, r_n).$$

Twierdzenie 2.1. *Istnieje macierz A wymiaru $n \times n$ o elementach z $\mathbb{F}_p$, zależna tylko od parametrów n, p oraz wartości $x_1, \ldots, x_n$, taka że*

$$R = S \cdot A.$$

Dowód. Przyjmijmy wektor

$$H = (h_0, \ldots, h_{t-1}, \ldots, h_{2(t-1)}, 0, \ldots, 0), \ |H| = n,$$

oraz

$$K = (h_0, \ldots, h_{t-1}, 0, \ldots, 0), \ |K| = n.$$

Niech $P : \mathbb{F}_p^n \to \mathbb{F}_p^{t-1}$ to rzut zachowujący pierwsze $t-1$ współrzędnych wektora:

$$P(\alpha_1, \ldots, \alpha_n) = (\alpha_1, \ldots, \alpha_{t-1}, 0, \ldots, 0).$$

Wtedy:

$$P(H) = P(h_0, \ldots, h_{t-1}, \ldots, h_{2(t-1)}, 0, \ldots, 0) = (h_0, \ldots, h_{t-1}, 0, \ldots, 0) = K,$$

$$H \cdot P = K. \tag{2.1}$$

Niech $B = (b_{i,j})$ będzie macierzą Vandermonda wymiaru $n \times n$, tzn. $b_{i,j} = x_j^{i-1}$ dla $i, j = 1, \ldots, n$.

$$B = \begin{bmatrix} x_1^0 & \cdots & x_n^0 \\ \vdots & \ddots & \vdots \\ x_1^{n-1} & \cdots & x_n^{n-1} \end{bmatrix}$$

Wtedy:

$$H \cdot B = S. \tag{2.2}$$

Wynika to z równości:

$$h(x_i) = h_0 \cdot x_i^0 + \ldots + h_{t-1} \cdot x_i^{t-1} + \ldots + h_{2(t-1)} \cdot x_i^{2(t-1)} + 0 \cdot x_i^{2t-1} + \ldots + 0 \cdot x_i^n = s_i.$$

Podobnie, jako że $k(x_i) = r_i$,

$$K \cdot B = R. \tag{2.3}$$

Podstawiając uzyskane równości (2.1), (2.3):

$$HP \cdot B = R.$$

Ponieważ B jest odwracalna (punkty x_i sa parami różne), korzystając z równania (2.2), otrzymujemy:

$$H(BB^{-1})PB = R,$$

$$S(B^{-1}PB) = R.$$

Stąd $A = B^{-1}PB$ to macierz o wartościach $a_{i,j}$, będącymi pewnymi stałymi z ciała $\mathbb{F}_p$, zależnymi jedynie od parametrów n, p i wybranych współrzędnych x_i. $\square$

Komunikacja między stronami

$$S \cdot A = [s_1, \ldots, s_n] \cdot \begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,n} \end{bmatrix} = [r_1, \ldots, r_n]$$

Strona P_i w ramach swojej części mnożenia $S \cdot A$ wykonuje dla $j = 1, \ldots, n$ mnożenie $s_i \cdot a_{i,j}$, czyli mnoży swój udział przez wartości z jednego wiersza (numer i) macierzy A. Natomiast wartości $r_i = k(x_i)$ są możliwe do obliczenia przez zsumowanie iloczynów wszystkich n wartości s_j z wartościami z jednej kolumny (numer i) macierzy A, tzn. $r_i = \sum_{j=1}^{n} s_j \cdot a_{j,i}$. Każda ze stron P_i po wykonaniu mnożenia $S \cdot A$ posiada po jednej części każdego r_j, dla $j = 1, \ldots, n$.

Aby każda ze stron P_i zgromadziła części r_i odpowiadające jej punktowi x_i, musi otrzymać je od pozostałych stron. Z tego powodu każda redukcja stopnia wielomianu wymaga jednej rundy wzajemnej komunikacji. Przez jedną rundę komunikacji rozumiemy tutaj wysłanie przez każdą stronę jednej wiadomości do każdej innej strony i odebranie od niej jednej wiadomości. Przyjmując, że strona wysyła wiadomość samej sobie, rundę komunikacji można zapisać jako macierz M wymiaru $n \times n$. Strona P_i wysyła rząd macierzy numer i, a odbiera kolumnę numer i.

$$M = \begin{bmatrix} s_1 \cdot a_{1,1} & s_1 \cdot a_{1,2} & \cdots & s_1 \cdot a_{1,n} \\ s_2 \cdot a_{2,1} & s_2 \cdot a_{2,2} & \cdots & s_2 \cdot a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ s_n \cdot a_{n,1} & s_n \cdot a_{n,2} & \cdots & s_n \cdot a_{n,n} \end{bmatrix}$$

Kiedy $n/4 \leqslant t - 1 < n/2$, po każdym mnożeniu konieczna jest redukcja stopnia wielomianu. Dlatego każdy iloczyn wymaga jednej rundy komunikacji.

2.3.2. Zapewnienie losowości współczynników wielomianu

Otrzymany w wyniku redukcji stopnia wielomian $k(x)$ spełnia warunek $k_0 = a \cdot b$, ale jego pozostałe współczynniki nie są wartościami losowymi. Jako iloczyn dwóch wielomianów, $h(x)$ nie może być niepodzielny, w przeciwieństwie do wielomianów zupełnie losowych. Zmniejszenie puli możliwych wartości współczynników h_i osłabia bezpieczeństwo ukrycia sekretu.

Należy wprowadzić dodatkową losowość do udziałów iloczynu wielomianów, żeby nie wynikały w prosty sposób ze współczynników czynników mnożenia. Wkład w zapewnienie losowości muszą mieć wszystkie strony. Opisana poniżej metoda zapewnienia losowości współczynników wielomianu ukrywającego sekret bazuje na pracy [WOG88].

Każda ze stron P_i ustala wielomian q_i stopnia $2(t-1)$, którego wyraz wolny jest równy 0, natomiast pozostałe współczynniki są liczbami losowymi. Odpowiada to podziałowi schematem Shamira sekretu równego 0 z progiem rekonstrukcji $2t-1$. Strona P_i generuje udziały $q_i(x_j)$ dla $j = 1, \ldots, n$ i przesyła udział $q_i(x_j)$ stronie P_j. Komunikację można przedstawić macierzą M', gdzie strona P_i wysyła rząd numer i, a odbiera kolumnę numer i.

$$M' = \begin{bmatrix} q_1(x_1) & q_1(x_2) & \cdots & q_1(x_n) \\ q_2(x_1) & q_2(x_2) & \cdots & q_2(x_n) \\ \vdots & \vdots & \ddots & \vdots \\ q_n(x_1) & q_n(x_2) & \cdots & q_n(x_n) \end{bmatrix}$$

Używając protokołu bezpiecznego dodawania sekretów, strony mogą dodać do siebie wielomiany $h(x)$ i $q_i(x)$. Przekształciwszy w ten sposób swoje udziały, strony otrzymają punkty wielomianu $\tilde{h}(x)$.

$$\tilde{h}(x) = h(x) + \sum_{j=1}^{n} q_j(x)$$

Spełnia on warunek $h_0 = a \cdot b$, ale wszystkie inne współczynniki mogą być dowolnymi liczbami z $\mathbb{F}_p$. Zależą one od wybranych przez strony wielomianów q_i i są nierozróżnialne od losowych.

Zapewniając w ten sposób losowość, do redukcji można użyć $\tilde{h}(x)$ zamiast $h(x)$, przyjmując $\tilde{s}_i = \tilde{h}(x_i)$ i wyliczając $\tilde{k}(x)$. Współczynniki wielomianu o zredukowanym stopniu będą nierozróżnialne od losowych [WOG88].

2.3.3. Mnożenie wielomianów

Podsumujemy teraz protokół mnożenia sekretów, którego zasada działania została opisana w poprzednich częściach.

Protokół 2.4. Mnożenie sekretów podzielonych schematem Shamira.
Dane:
- $t - 1 < n/2$,
- $[a] = \{f(x_1), \ldots, f(x_n)\}$ - udziały sekretu ukrywanego przez wielomian $f(x)$ (zachodzi zatem $f(0) = a$),
- $[b] = \{g(x_1), \ldots, g(x_n)\}$ - udziały sekretu ukrywanego przez wielomian $g(x)$ (zachodzi zatem $g(0) = b$).

Przygotowanie:
1. Przed rozpoczęciem działań każda ze stron może obliczyć macierz $A = B^{-1}PB$, gdzie
$$B = (b_{i,j}) : \; b_{i,j} = x_j^{i-1} \text{ dla } i, j = 1, \ldots, n,$$

$$P(\alpha_1, \ldots, \alpha_n) = (\alpha_1, \ldots, \alpha_{t-1}, 0, \ldots, 0).$$

Przebieg:

1. Każda ze stron P_i oblicza udział

$$h(x_i) = a_i \cdot b_i \ (mod\ p).$$

2. Każda ze stron P_i dokonuje podziału schematem Shamira sekretu równego 0. Używa do tego wielomianu stopnia $2(t-1)$ oznaczanego $q_i(x)$, dla którego $q_i(0) = 0$.
3. Każda ze stron P_i oblicza n udziałów na podstawie $q_i(x)$ i przesyła udział $q_i(x_j)$ stronie P_j.
4. Każda ze stron P_i oblicza udział

$$\tilde{s}_i = \tilde{h}(x_i) = h(x_i) + \sum_{j=1}^{n} q_j(x_i).$$

5. Każda ze stron P_i przesyła część $\tilde{s}_i \cdot a_{i,j}$ stronie P_j.
6. Każda ze stron P_i oblicza udział

$$\tilde{r}_i = \tilde{k}(x_i) = \sum_{j=1}^{n} \tilde{s}_j \cdot a_{j,i}.$$

Wyjście:

— $[c] = \{\tilde{r}_1, \ldots, \tilde{r}_n\}$ - udziały sekretu ukrywanego przez wielomian $\tilde{k}(x)$ stopnia $t-1$, dla którego $\tilde{k}(0) = f(0) \cdot g(0) = a \cdot b$.

Naprawa błędów mnożenia

W przypadku $n/3 \leqslant t-1 < n/2$ nie jest możliwa naprawa żadnych błędów mnożenia. Jeżeli choć jedna strona poda nieprawidłowy udział, rekonstrukcja iloczynu zwróci pewne $c \neq a \cdot b$. Nawet jeśli nieuczciwe strony współpracują ze sobą, nie mają pełnej kontroli nad c. Jednak uczciwe strony nie będą mogły mieć pewności, żę mnożenie przebiegło poprawnie.

W przypadku $t-1 < n/4$ możliwe jest zastosowanie właściwości kodów korekcyjnych bezpośrednio na $h(x)$ i poprawienie $t-1$ błędów mnożenia na wielomianie stopnia $2(t-1)$ [WOG88]. Kiedy $t-1 < n/3$, liczba udziałów nie jest wystarczająca do naprawy $t-1$ błędów na wielomianie stopnia $2(t-1)$. Można jednak zastosować dodatkowe techniki, aby naprawić błędy na wielomianach stopnia $t-1$, a jednocześnie zweryfikować poprawność punktów używanych do redukcji. Wymagane jest przesłanie dodatkowych danych pomiędzy stronami. Techniki te nie zostały wykorzystane w projekcie *Aukciszek*, ich opis zawiera praca [WOG88].

Przykład 2.9. Mnożenie sekretów.
Dane:
— $n = 5$, $t = 2$,
— $p = 23$,
— $(x_1, \ldots, x_n) = (1, \ldots n)$.
Wzory wielomianów ukrywających sekrety są znane tylko osobie dzielącej sekrety:
— $f(x) = 3 + 20x$,
— $f(0) = a = 3$,
— $g(x) = 4 + 21x$,
— $g(0) = b = 4$.
Wejście:
— $[a] = \{f(1), \ldots, f(n)\} = \{0, 20, 17, 14, 11\}$,
— $[b] = \{g(1), \ldots, g(n)\} = \{2, 0, 21, 19, 17\}$.
Przygotowanie:
1. Każda ze stron oblicza macierz $A = B^{-1}PB$.

$$B = \begin{bmatrix} 1^0 & 2^0 & 3^0 & 4^0 & 5^0 \\ 1^1 & 2^1 & 3^1 & 4^1 & 5^1 \\ 1^2 & 2^2 & 3^2 & 4^2 & 5^2 \\ 1^3 & 2^3 & 3^3 & 4^3 & 5^3 \\ 1^4 & 2^4 & 3^4 & 4^4 & 5^4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 9 & 16 & 25 \\ 1 & 8 & 27 & 64 & 125 \\ 1 & 16 & 81 & 256 & 625 \end{bmatrix}$$

$$B \equiv \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 9 & 16 & 2 \\ 1 & 8 & 4 & 18 & 10 \\ 1 & 16 & 12 & 3 & 4 \end{bmatrix} \ (mod\ 23)$$

$$B^{-1} \equiv \begin{bmatrix} 5 & 7 & 2 & 9 & 1 \\ 13 & 14 & 17 & 6 & 19 \\ 10 & 15 & 18 & 20 & 6 \\ 18 & 14 & 20 & 21 & 19 \\ 1 & 19 & 12 & 13 & 1 \end{bmatrix} \ (mod\ 23)$$

$$A = B^{-1}PB = \begin{bmatrix} 5 & 7 & 2 & 9 & 1 \\ 13 & 14 & 17 & 6 & 19 \\ 10 & 15 & 18 & 20 & 6 \\ 18 & 14 & 20 & 21 & 19 \\ 1 & 19 & 12 & 13 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 9 & 16 & 2 \\ 1 & 8 & 4 & 18 & 10 \\ 1 & 16 & 12 & 3 & 4 \end{bmatrix}$$

$$A \equiv \begin{bmatrix} 12 & 19 & 3 & 10 & 17 \\ 4 & 18 & 9 & 0 & 14 \\ 2 & 17 & 9 & 1 & 16 \\ 9 & 0 & 14 & 5 & 19 \\ 20 & 16 & 12 & 8 & 4 \end{bmatrix} \ (mod\ 23)$$

Przebieg:

1. Każda ze stron oblicza swój udział $h(x)$:
   — $h(1) = f(1)g(1) = 0 \cdot 2 \equiv 0\ (mod\ 23)$,
   — $h(2) = f(2)g(2) = 20 \cdot 0 \equiv 0\ (mod\ 23)$,
   — $h(3) = f(3)g(3) = 17 \cdot 21 = 357 \equiv 12\ (mod\ 23)$,
   — $h(4) = f(4)g(4) = 14 \cdot 19 = 266 \equiv 13\ (mod\ 23)$,
   — $h(4) = f(5)g(5) = 11 \cdot 17 = 187 \equiv 3\ (mod\ 23)$.
2. Każda ze stron P_i dokonuje podziału sekretu wartości 0 schematem Shamira. Używa do tego wielomianu stopnia $2(t-1)$ oznaczanego $q_i(x)$, dla którego $q_i(0) = 0$:
   — $q_1(x) = 0 + 1x + 2x^2$,
   — $q_2(x) = 0 + 2x + 1x^2$,
   — $q_3(x) = 0 + 3x + 3x^2$,
   — $q_4(x) = 0 + 2x + 2x^2$,
   — $q_5(x) = 0 + 4x + 5x^2$.
3. Każda ze stron oblicza n udziałów na podstawie $q_i(x)$:
   — $q_1(1) = 3,\quad q_1(2) = 10,\ q_1(3) = 21,\ q_1(4) = 13,\ q_1(5) = 9$,
   — $q_2(1) = 3,\quad q_2(2) = 8,\quad q_2(3) = 15,\ q_2(4) = 1,\quad q_2(5) = 12$,
   — $q_3(1) = 6,\quad q_3(2) = 18,\ q_3(3) = 13,\ q_3(4) = 14,\ q_3(5) = 21$,
   — $q_4(1) = 4,\quad q_4(2) = 12,\ q_4(3) = 1,\quad q_4(4) = 17,\ q_4(5) = 14$,
   — $q_5(1) = 9,\quad q_5(2) = 5,\quad q_5(3) = 11,\ q_5(4) = 4,\quad q_5(5) = 7$.
4. Każda ze stron przesyła udział $q_i(x_j)$ stronie P_j. Komunikację stron można przedstawić za pomocą macierzy M'.

$$M' = \begin{bmatrix} 3 & 10 & 21 & 13 & 9 \\ 3 & 8 & 15 & 1 & 12 \\ 6 & 18 & 13 & 14 & 21 \\ 4 & 12 & 1 & 17 & 14 \\ 9 & 5 & 11 & 4 & 7 \end{bmatrix}$$

5. Każda ze stron oblicza swój udział $\tilde{s}_i = \tilde{h}(x_i) = h(x_i) + q_1(x_i) + \ldots + q_n(x_i)$:
   — $\tilde{s}_1 = \tilde{h}(1) = 0 + 3 + 3 + 6 + 4 + 9 = 25 \equiv 2\ (mod\ 23)$,
   — $\tilde{s}_2 = \tilde{h}(2) = 0 + 10 + 8 + 18 + 12 + 5 = 53 \equiv 7\ (mod\ 23)$,
   — $\tilde{s}_3 = \tilde{h}(3) = 12 + 21 + 15 + 13 + 1 + 11 = 73 \equiv 4\ (mod\ 23)$,

— $\tilde{s}_4 = \tilde{h}(4) = 13 + 13 + 1 + 14 + 17 + 4 = 62 \equiv 16 \ (mod\ 23)$,
— $\tilde{s}_5 = \tilde{h}(5) = 3 + 9 + 12 + 21 + 14 + 7 = 66 \equiv 20 \ (mod\ 23)$.

6. Każda ze stron oblicza n części $\tilde{s}_i \cdot a_{i,1}, \ldots, \tilde{s}_i \cdot a_{i,n}$.

$$\begin{bmatrix} 2 \cdot 12 & 2 \cdot 19 & 2 \cdot 3 & 2 \cdot 10 & 2 \cdot 17 \\ 7 \cdot 4 & 7 \cdot 18 & 7 \cdot 9 & 7 \cdot 0 & 7 \cdot 14 \\ 4 \cdot 2 & 4 \cdot 17 & 4 \cdot 9 & 4 \cdot 1 & 4 \cdot 16 \\ 16 \cdot 9 & 16 \cdot 0 & 16 \cdot 14 & 16 \cdot 5 & 16 \cdot 19 \\ 20 \cdot 20 & 20 \cdot 16 & 20 \cdot 12 & 20 \cdot 8 & 20 \cdot 4 \end{bmatrix}$$

7. Każda ze stron P_i przesyła część $\tilde{s}_i \cdot a_{i,j}$ stronie P_j. Komunikacje można przedstawić za pomocą macierzy M.

$$M \equiv \begin{bmatrix} 1 & 15 & 6 & 20 & 11 \\ 5 & 11 & 17 & 0 & 6 \\ 8 & 22 & 13 & 4 & 18 \\ 6 & 0 & 17 & 11 & 5 \\ 9 & 21 & 10 & 22 & 11 \end{bmatrix} \ (mod\ 23)$$

8. Każda ze stron P_i oblicza udział $\tilde{r}_i = \tilde{k}(x_i) = \tilde{s}_1 \cdot a_{1,i} + \ldots + \tilde{s}_n \cdot a_{n,i}$:
— $\tilde{r}_1 = \tilde{k}(1) = 1 + 5 + 8 + 6 + 9 = 29 \equiv 6 \ (mod\ 23)$,
— $\tilde{r}_2 = \tilde{k}(2) = 15 + 11 + 22 + 0 + 21 = 69 \equiv 0 \ (mod\ 23)$,
— $\tilde{r}_3 = \tilde{k}(3) = 6 + 17 + 13 + 17 + 10 = 63 \equiv 17 \ (mod\ 23)$,
— $\tilde{r}_4 = \tilde{k}(4) = 20 + 0 + 4 + 11 + 22 = 57 \equiv 11 \ (mod\ 23)$,
— $\tilde{r}_5 = \tilde{k}(5) = 11 + 6 + 18 + 5 + 11 = 51 \equiv 5 \ (mod\ 23)$.

Wyjście:
— $[c] = \{\tilde{r}_1, \ldots, \tilde{r}_n\} = \{6, 0, 17, 11, 5\}$ - udziały sekretu ukrywanego przez wielomian $\tilde{k}(x)$ stopnia $t - 1$, dla którego $\tilde{k}(0) = f(0) \cdot g(0) = 3 \cdot 4 = 12$.

Sprawdzenie:
— $h(x) = f(x) \cdot g(x) = 12 + 143x + 420x^2 \equiv 12 + 5x + 6x^2 \ (mod\ 23)$,
— $\tilde{h}(x) = (12 + 5x + 6x^2) + (0 + 1x + 2x^2) + (0 + 2x + 1x^2) + (0 + 3x + 3x^2) + (0 + 2x + 2x^2) + (0 + 4x + 5x^2) \equiv 12 + 17x + 19x^2 \ (mod\ 23)$,
— $\tilde{k}(x) = 12 + 17x$,
— $\tilde{k}(1) = 29, \quad \tilde{k}(2) = 46, \quad \tilde{k}(3) = 63, \quad \tilde{k}(4) = 80, \quad \tilde{k}(5) = 97$,
— $\{\tilde{k}(1), \ldots, \tilde{k}(n)\} \equiv \{6, 0, 17, 11, 5\} \ (mod\ 23)$.

2.3.4. Ograniczenie liczby rund komunikacji

W systemie *Aukciszek* używane jest rozwiązanie opisane w protokole 2.4. Istnieje jednak alternatywa ograniczająca komunikację do jednej rundy.

W protokole 2.4 strony wykonują dwie rundy komunikacji, zapisane macierzami M' i M.

$$M' = \begin{bmatrix} q_1(x_1) & q_1(x_2) & \cdots & q_1(x_n) \\ q_2(x_1) & q_2(x_2) & \cdots & q_2(x_n) \\ \vdots & \vdots & \ddots & \vdots \\ q_n(x_1) & q_n(x_2) & \cdots & q_n(x_n) \end{bmatrix}$$

$$M = \begin{bmatrix} \tilde{s}_1 \cdot a_{1,1} & \tilde{s}_1 \cdot a_{1,2} & \cdots & \tilde{s}_1 \cdot a_{1,n} \\ \tilde{s}_2 \cdot a_{2,1} & \tilde{s}_2 \cdot a_{2,2} & \cdots & \tilde{s}_2 \cdot a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{s}_n \cdot a_{n,1} & \tilde{s}_n \cdot a_{n,2} & \cdots & \tilde{s}_n \cdot a_{n,n} \end{bmatrix}$$

$$\text{gdzie } \tilde{s}_i = \tilde{h}(x_i) = h(x_i) + \sum_{j=1}^{n} q_j(x_i).$$

Strony najpierw wymieniają się wartościami macierzy M'. Następnie obliczają $\tilde{h}(x) = h(x) + \sum_{j=1}^{n} q_j(x)$, przed wyliczeniem i przesłaniem M. Jednak tę sumę można rozbić na dwie części:

$$\tilde{h}(x) = \Big(h(x) + q_i(x)\Big) + \sum_{\substack{j=1 \\ j \neq i}}^{n} q_j(x).$$

Korzystając z właściwości protokołu dodawania sekretów, strona P_i może obliczyć udział sumy $h(x) + q_i(x)$ jako $h(x_i) + q_i(x_i)$. Może to zrobić przed wymianą udziałów pozostałych wielomianów $q_j(x)$.

W protokole 2.4 strony wysyłają wiadomości postaci $\tilde{s}_i \cdot a_{i,j}$. Okazuje się, że można pominąć część obliczeń i zmienić kolejność działań.

$$\tilde{s}_i \cdot a_{i,j} = \Big(h(x_i) + \sum_{j=1}^{n} q_j(x_i)\Big) \cdot a_{i,j} = \Big(h(x_i) \cdot a_{i,j}\Big) + \sum_{j=1}^{n} q_j(x_i) \cdot a_{i,j}$$

Wielomiany $q_j(x)$ ukrywają sekret 0, tzn. ich wyrazy wolne są równe 0. Mnożenie tych wielomianów przez stałe może zmieniać pozostałe współczynniki, ale wyrazy wolne pozostaną zerami. Nie będzie to miało wpływu na rekonstrukcję sekretu. Dlatego mnożenie wielomianów $q_j(x)$ przez stałe $a_{i,j}$ można pominąć. Strona P_i może przesłać wiadomość postaci $h(x_i) \cdot a_{i,j} + q_i(x_i)$, do której później zostanie dodana suma $\sum_{j=1, j \neq i}^{n} q_j(x_i)$. Komunikację między stronami można zapisać jedną macierzą M''.

$$M'' = \begin{bmatrix} h(x_1) \cdot a_{1,1} + q_1(x_1) & h(x_1) \cdot a_{1,2} + q_1(x_2) & \cdots & h(x_1) \cdot a_{1,n} + q_1(x_n) \\ h(x_2) \cdot a_{2,1} + q_2(x_1) & h(x_2) \cdot a_{2,2} + q_2(x_2) & \cdots & h(x_2) \cdot a_{2,n} + q_2(x_n) \\ \vdots & \vdots & \ddots & \vdots \\ h(x_n) \cdot a_{n,1} + q_n(x_1) & h(x_n) \cdot a_{n,2} + q_n(x_2) & \cdots & h(x_n) \cdot a_{n,n} + q_n(x_n) \end{bmatrix}$$

Sumując kolumnę numer i macierzy M'', otrzymujemy:

$$\sum_{j=1}^{n} h(x_j) \cdot a_{j,i} + q_j(x_i) = \sum_{j=1}^{n} h(x_j) \cdot a_{j,i} + \sum_{j=1}^{n} q_j(x_i) = k(x_i) + q(x_i) = \tilde{k}(x_i).$$

Dlatego, otrzymując kolumnę numer i, strona P_i otrzymuje swój udział iloczynu.

Udziały mniej bezpiecznego (o nie w pełni losowych współczynnikach) wielomianu $h(x)$ nie są przesyłane jawnie. Są „zaciemnione" przez dodanie do nich udziałów wielomianów $q_i(x)$.

2.3.5. Złożoność obliczeniowa mnożenia

Każda z n stron mnoży swoje udziały $f(x_i)$ i $g(x_i)$, otrzymując $h(x_i)$. Dokonuje podziału sekretu równego 0 pomiędzy n stron, używając wielomianu $q_i(x)$ stopnia $t-1$. Mnoży swój udział $h(x_i)$ przez $a_{i,j}$ i dodaje do wyniku udział $q_i(x_j)$, dla $j = 1, \ldots, n$. Wyniki przesyła stronom P_j, dla $j = 1, \ldots, n$. Strona P_i, po otrzymaniu n komunikatów od wszystkich stron, sumuje je, otrzymując udział iloczynu $\tilde{k}(x_i)$. Dlatego złożoność mnożenia sekretów to $n \cdot (O(1) + O(n \cdot (t-1)) + O(n) + O(n) + O(n)) = n \cdot O(n \cdot (t-1)) = O(n^2 \cdot (t-1))$.

Przy każdym mnożeniu wymagana jest jedna runda wzajemnej komunikacji pomiędzy n stronami.

Rozdział 3

Porównanie sekretów

3.1. Porównanie liczb na platformie Aukciszek

Naszym celem jest bezpieczne porównanie podzielonych sekretów. Wynik musi zdradzać jedynie informację, która z porównywanych liczb jest większa. Porównanie nie może ujawnić żadnych informacji o porównywanych liczbach ani o wartości różnicy między nimi.

Wykorzystywany w projekcie *Aukciszek* protokół jest wzorowany na zastosowanej w praktyce metodzie duńskich naukowców, opisanej w pracy [Bog+09] i książce [CDN15]. Wykorzystuje wcześniej wprowadzone operacje dodawania i mnożenia sekretów. Do porównania liczb korzysta z reprezentacji binarnej.

Zakładamy, że pomiędzy n stron podzielone są sekrety $a, b \in \mathbb{F}_p$ zgodnie ze schematem Shamira z progiem rekonstrukcji t, gdzie $t - 1 < n/2$. Celem protokołu jest otrzymanie nowego podziału sekretu, który będzie ukrywał wartość 1, jeżeli $a \geqslant b$, albo 0, jeżeli $a < b$.

Przyjmujemy, że maksymalna liczba bitów reprezentacji dwójkowej a, b to l. Liczbę bitów, na których zapisana jest liczba pierwsza p, oznaczamy jako m. Zakładamy, że m jest większe lub równe l.

Protokół opiera się na analizie różnicy $a - b$ bez rekonstrukcji jej wartości. Znajomość różnicy ujawniałaby stronie znającej jedną liczbę dokładną wartość drugiej liczby. Wystarczy stwierdzić, czy różnica jest ujemna ($a < b$). Aby to zrobić, można przetworzyć bity jej reprezentacji binarnej. Jeżeli po dodaniu do $a - b$ liczby 2^l najbardziej znaczący bit wyniku jest równy 0, różnica $a - b$ jest ujemna. W przeciwnym wypadku musi być ona większa lub równa zeru. Wyliczając podział tego bitu, uzyskuje się podział wyniku porównania, który można zrekonstruować bez ujawniania dodatkowych informacji.

W tej części przedstawione zostaną protokoły działające na podziałach bi-

tów, służące do wyizolowania i analizy podziału bitu oznaczającego wynik porównania.

3.1.1. Bezpieczny XOR

Operując na podziałach sekretów, nie można w łatwy sposób wykonać redukcji sekretu modulo 2. Dlatego wynik $[a] \oplus [b]$ oblicza się jako $[a] + [b] - 2[a] \cdot [b]$. Jeżeli a, b to wartości ze zbioru $\{0, 1\}$, wtedy wynik obliczeń również jest wartością z tego zbioru.

a	b	$a+b$	$a \cdot b$	$2 \cdot (a \cdot b)$	$a+b-2 \cdot (a \cdot b)$
0	0	0	0	0	0
0	1	1	0	0	1
1	0	1	0	0	1
1	1	2	1	2	0

Obliczenie w ten sposób funkcji XOR wymaga jednego mnożenia sekretów, czyli jednej rundy komunikacji pomiędzy stronami.

3.1.2. Podział losowego bitu

Naszym celem jest otrzymanie podziału sekretu, ukrywającego wartość zmiennej losowej o wartości ze zbioru $\{0, 1\}$ i rozkładzie jednostajnym. Wkład w generowanie udziałów sekretu powinny mieć wszystkie strony.

Proste rozwiązanie zakłada, że każda z n stron dzieli wybrany przez siebie bit pomiędzy wszystkie strony. Obliczywszy funkcję XOR tak powstałych bitów, strony otrzymują udziały losowego bitu. Jednak obliczenie $n-1$ razy funkcji XOR wymaga $n-1$ rund komunikacji. Dodatkowo, jeżeli jedna ze stron użyje podziału sekretu niebędącego bitem, wynik może nie być bitem [CDN15, (9.1.6)].

Alternatywą jest skorzystanie z faktu, że w ciele $\mathbb{F}_p$ każdy niezerowy element u posiada element przeciwny $-u \neq 0$. Dla każdego $u \in \mathbb{F}_p/\{0\}$ zarówno u^2 jak i $(-u)^2$ są równe $u \cdot u \neq 0$. Losując element u z rozkładu jednostajnego, prawdopodobieństwo, że znajduje się on w przedziale $[1; \frac{p}{2}]$, wynosi $\frac{1}{2}$. Na podstawie potęgi $v \equiv u^2 \ (mod\ p)$ można algorytmicznie wyznaczyć pierwiastek $w \in [1; \frac{p}{2}]$, $w^2 \equiv v \ (mod\ p)$. Jeżeli $u \in [1; \frac{p}{2}]$, wtedy $w = u$, a $w^{-1}u = 1$. W przeciwnym wypadku $w = -u$, a $w^{-1}u = -1$. Obie sytuacje są jednakowo prawdopodobne, dlatego znak iloczynu $w^{-1} \cdot u$ będzie oznaczał wartość losowego bitu.

Protokół 3.1. Generowanie podziału losowego bitu [CDN15, (9.1.6 Protocol RandomBit)].
Dane:
— $P_1, \ldots, P_n$ - n stron, pomiędzy które dzielony jest sekret,
— $t - 1 < n/2$, gdzie t to próg rekonstrukcji sekretu,
— p - liczba pierwsza.
Przebieg:
1. Każda ze stron P_i dokonuje podziału sekretu losowej wartości $u_i \in \mathbb{F}_p$ pomiędzy n stron. Używa schematu Shamira z progiem rekonstrukcji t.
2. Strony wykonują bezpieczne dodawanie sekretów, otrzymując podział losowej liczby $u \in \mathbb{F}_p$.
$$[u] = [u_1] + \ldots + [u_n]$$
3. Strony wykonują bezpieczne mnożenie sekretów, otrzymując podział $v = u^2 \ (mod\ p)$.
$$[v] = [u] \cdot [u]$$
4. Strony dokonują rekonstrukcji v. Każda ze stron poznaje wartość v.
5. Jeżeli $v = 0$, strony powtarzają protokół.
6. Jeżeli $v > 0$, strony jawnie wyznaczają najmniejszy pierwiastek kwadratowy v, oznaczany jako w. Wszystkie strony poznają wartość w^{-1}.
7. Strony wykonują bezpieczne mnożenie sekretu u przez stałą w^{-1}, otrzymując z równym prawdopodobieństwem podział wartości 1 albo -1, oznaczany jako $[f]$.
$$[f] = w^{-1} \cdot [u]$$
8. Dodając do wyniku 1 i dzieląc przez 2, strony otrzymują podział losowego bitu.
$$[r_b] = ([f] + 1) \cdot 2^{-1}$$

Wyjście:
— $[r_b]$ - podział wartości zmiennej losowej r_b o wartościach ze zbioru $\{0, 1\}$ i rozkładzie jednostajnym.

Protokół wymaga tylko jednego mnożenia sekretów, a zatem tylko jednej rundy komunikacji. Jeżeli choć jedna z wartości u_i jest losowa, wynik jest losowym bitem.

Protokół należy powtórzyć, kiedy $v = 0$. Dzieje się tak jedynie wtedy, gdy $u = 0$. Jako że $u \in \mathbb{F}_p$ jest losowane z rozkładu jednostajnego, prawdopodobieństwo tego zdarzenia jest równe $\frac{1}{p}$. Przy określaniu złożoności kolejnych protokołów będziemy je pomijać, przyjmując, że podział losowego bitu wymaga jednej rundy komunikacji.

3.1.3. Analiza znaku różnicy sekretów

Aby porównać liczby, wystarczy poznać znak ich różnicy. Opisana poniżej metoda jego wyznaczania pochodzi z pracy [Bog+09, (4.1)].

Używana reprezentacja binarna nie pozwala na zapis liczb ujemnych. Dlatego do różnicy dodaje się 2^l, gdzie l to liczba bitów reprezentacji binarnej porównywanych liczb. Jeżeli różnica jest nieujemna, najbardziej znaczący bit numer l nie zmieni się i pozostanie jedynką. W przeciwnym przypadku będzie równy zero. Naszym celem jest bezpieczna analiza bitu numer l liczby postaci

$$2^l + a - b,$$

która przyjmuje wartości z zakresu $[1; 2^{l+1} - 1]$.

Niech l to liczba bitów reprezentacji binarnej liczb a, b. Niech $k \geqslant 1$ to parametr bezpieczeństwa. Niech p to liczba pierwsza zapisana na $l + k + 2$ bitach. Niech $[r]$ to podział losowej liczby r, otrzymany na podstawie podziałów losowych bitów $[r_{l+k}], \ldots, [r_0]$, tzn.

$$[r] = \sum_{i=0}^{l+k} 2^i \cdot [r_i].$$

Liczba r może być zapisana na $l + k + 1$ bitach, dlatego $r \in \mathbb{Z}_p$. Definiujemy liczbę d, której podział można obliczyć poprzez bezpieczne dodawanie sekretów.

$$[d] = 2^{l+k+1} - [r] + 2^l + [a] - [b]$$

Maksymalna wartość d to $2^{l+k+1} - 0 + 2^l + (2^l - 1) - 0 = 2^{l+k+1} + 2^{l+1} - 1$, a minimalna to $2^{l+k+1} - (2^{l+k+1} - 1) + 2^l + 0 - (2^l - 1) = 2$. Aby można było dokonać podziału każdego d, zakładamy, że $p \geqslant 2^{l+k+1} + 2^{l+1}$. Wtedy zawsze $d \in \mathbb{Z}_p$.

Wtedy bity liczby d to bity wyrażenia $2^l + a - b$ „zaciemnione" bitami liczby r. Dodając do nich bity r_i, otrzymuje się bity sumy $d + r$. Nie potrzeba obliczać wszystkich bitów tej sumy, wystarczy poznać bit numer l oznaczany jako w, kodujący znak różnicy $a - b$. Jego wartość zależy od bitów d_l, r_l oraz bitu przeniesienia na pozycji numer l przy dodawaniu $d + r$, oznaczanego jako Z.

$$[w] = d_l \oplus [r_l] \oplus [Z]$$

3.1.4. Wyznaczenie wartości przeniesienia

Zakładamy, że znamy bity liczby d równej $2^{l+k+1} - r + 2^l + a - b$. Mamy do dyspozycji również podziały bitów $[r_i]$. Naszym celem jest wyznaczenie podziału oznaczającego bit przeniesienia na pozycji numer l przy dodawaniu $d + r$.

Protokół 3.2. Wyznaczenie podziału wartości przeniesienia przy dodawaniu liczb na podstawie podziałów bitów [Bog+09, (4.1)].
Dane:
— p - liczba pierwsza,
— l - liczba bitów reprezentacji binarnej porównywanych liczb,
— k - parametr bezpieczeństwa,
— $l + k + 2$ - liczba bitów reprezentacji binarnej liczby p.
Wejście:
— $d_{l+k+2}, \ldots, d_0$ - bity liczby d, równej $2^{l+k+1} - r + 2^l + a - b$,
— $[r_{l+k}], \ldots, [r_0]$ - podziały bitów losowej liczby r.
Przebieg:
1. Strony obliczają wyrażenie:

$$\begin{pmatrix} [z] \\ [Z] \end{pmatrix} = \begin{pmatrix} d_{l-1} \oplus [r_{l-1}] \\ d_{l-1} \end{pmatrix} \diamond \cdots \diamond \begin{pmatrix} d_0 \oplus [r_0] \\ d_0 \end{pmatrix} \diamond \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

gdzie:

$$\begin{pmatrix} x \\ X \end{pmatrix} \diamond \begin{pmatrix} y \\ Y \end{pmatrix} = \begin{pmatrix} x \wedge y \\ x \wedge (X \oplus Y) \oplus X \end{pmatrix}.$$

Wyjście:
— $[Z]$ - podział wartości przeniesienia na pozycji numer l.

Jeżeli bity d_i i r_i są różne, bit przeniesienia, oznaczany jako c_i, jest równy poprzedniemu, tzn. $c_i = c_{i-1}$. W przeciwnym wypadku przyjmuje on wartość dodawanych bitów, tzn. $c_i = d_i = r_i$. Aby rozpocząć obliczenia, za bit przeniesienia c_{-1} przyjmuje się stałą 0.

Lemat 3.1. *Działanie $\diamond$ jest łączne.*

Pomijamy dowód lematu. Można go przeprowadzić przez sprowadzenie formuł logicznych do koniunkcyjnej postaci normalnej.

Twierdzenie 3.2. *Obliczenie wyrażenia*

$$\begin{pmatrix} z \\ Z \end{pmatrix} = \begin{pmatrix} d_l \oplus r_l \\ d_l \end{pmatrix} \diamond \cdots \diamond \begin{pmatrix} d_0 \oplus r_0 \\ d_0 \end{pmatrix} \diamond \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

pozwala na poznanie bitu przeniesienia c_l przy dodawaniu bitów d_l i r_l liczb d i r. Wartość Z jest równa c_l.

Dowód. Obliczenie wartości przeniesienia na pozycji 0 to poznanie wartości $\binom{z_0}{Z_0}$.

$$\begin{pmatrix} z_0 \\ Z_0 \end{pmatrix} = \begin{pmatrix} d_0 \oplus r_0 \\ d_0 \end{pmatrix} \diamond \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} d_0 \oplus r_0 \\ d_0 \end{pmatrix} \diamond \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} (d_0 \oplus r_0) \wedge 0 \\ (d_0 \oplus r_0) \wedge (d_0 \oplus 0) \oplus d_0 \end{pmatrix} = \begin{pmatrix} 0 \\ (d_0 \oplus r_0) \wedge d_0 \oplus d_0 \end{pmatrix}$$

Jeżeli $d_0 \neq r_0$, wartość Z_0 wynosi $1 \wedge d_0 \oplus d_0 = 0$, czyli jest równa $c_{-1} = 0$. W przeciwnym wypadku przyjmuje ona wartość $0 \wedge d_0 \oplus d_0 = d_0$, czyli wartość dodawanych bitów.

Załóżmy, że obliczyliśmy wartość przeniesienia na pozycji $i-1$, tzn. znamy wartość

$$\binom{y}{Y} = \binom{d_{i-1} \oplus r_{i-1}}{d_{i-1}} \diamond \cdots \diamond \binom{d_0 \oplus r_0}{d_0} \diamond \binom{0}{0},$$

gdzie $Y = c_{i-1}$.
Niech

$$\binom{z}{Z} = \binom{x}{X} \diamond \binom{y}{Y} = \binom{d_i \oplus r_i}{d_i} \diamond \binom{y}{Y}.$$

Jeżeli bity d_i i r_i są różne, działanie $\binom{x}{X} \diamond \binom{y}{Y}$ zwraca wynik równy $\binom{y}{Y}$. W przeciwnym wypadku zwracany jest wynik $\binom{x}{X}$.

d_i	r_i	$d_i \oplus r_i$	$x \wedge y$	$X \oplus Y$	$x \wedge (X \oplus Y)$	$x \wedge (X \oplus Y) \oplus X$
0	0	0	0	Y	0	0
0	1	1	y	Y	Y	Y
1	0	1	y	$\neg Y$	$\neg Y$	Y
1	1	0	0	$\neg Y$	0	1

Zatem, jeżeli $d_0 \neq r_0$, wartość Z wynosi Y, czyli jest równa c_{i-1}. W przeciwnym wypadku przyjmuje ona wartość $X = d_0$, czyli wartość dodawanych bitów. □

Wymagana liczba rund komunikacji

Przed rozpoczęciem obliczeń konieczne jest wygenerowanie podziałów $l+k+1$ losowych bitów liczby r wymagające $l+k+1$ rund komunikacji. Wynikiem działania $\binom{[x]}{X} \diamond \binom{0}{0}$ jest $\binom{[y]}{[Y]}$. Nie wymaga ono komunikacji. Wynikiem działania $\binom{[x]}{X} \diamond \binom{[y]}{[Y]}$ jest $\binom{[z]}{[Z]}$. Wymaga ono trzech rund komunikacji. Takie działanie wykonuje się $l-1$ razy, co wymaga $3(l-1)$ rund komunikacji. Stąd wyznaczenie wartości przeniesienia wymaga $(l+k+1)+3(l-1) = 4l+k$ rund komunikacji.

3.1.5. Porównanie liczb na podstawie ich podziałów

Protokół 3.3. Porównanie liczb na podstawie ich podziałów [Bog+09, (4.1)].
Parametry:
- p - liczba pierwsza,
- l - liczba bitów reprezentacji binarnej liczb a, b,
- k - parametr bezpieczeństwa,
- $l+k+2$ - liczba bitów reprezentacji binarnej liczby p.

Wejście:
- $[a]$ - podział liczby a,

— $[b]$ - podział liczby b.

Przebieg:

1. Strony $l+k+1$ razy wykonują protokół podziału losowego bitu, otrzymując podziały:
$$[r_{l+k}], \ldots, [r_0],$$
$$[r] = \sum_{i=0}^{l+k} 2^i \cdot [r_i].$$
2. Strony wykonują bezpieczne dodawanie i odejmowanie, otrzymując podział liczby d.
$$[d] = 2^{l+k+1} - [r] + 2^l + [a] - [b]$$
3. Strony dokonują rekonstrukcji liczby d. Każda ze stron poznaje jej bity.
$$d_{l+k+2}, \ldots, d_0$$
4. Strony wykonują protokół wyznaczenia podziału wartości przeniesienia. Otrzymują podział bitu przeniesienia na pozycji numer l przy dodawaniu $d+r$ na podstawie podziałów bitów $[r_i]$, oznaczany $[Z]$.
5. Strony obliczają podział wyniku dodawania $d+r$ na pozycji numer l. Otrzymują podział wyniku porównania.
$$[w] = d_l \oplus [r_l] \oplus [Z]$$
6. Strony dokonują rekonstrukcji $[w]$. Każda ze stron poznaje wartość bitu w.

Wyjście:

— w - bit oznaczający wynik porównania. Jeżeli $a \geqslant b$, wtedy $w = 1$. W przeciwnym przypadku $w = 0$.

3.1.6. Równoległe wykonywanie wielu rund komunikacji

Wykorzystywane sposoby przekazu informacji mogą wprowadzać niebanalne koszty komunikacji związane z czasem nawiązywania połączenia, jego utrzymaniem, optymalnym rozmiarem pakietu itp. Korzystne może okazać się łączenie rund komunikacji w zbiory, które mogą być wykonane równolegle. W zależności od implementacji przekazu informacji może to zmniejszyć jego koszty.

Wykonanie działania $\binom{[z]}{[Z]} = \binom{[x]}{[X]} \diamond \binom{[y]}{[Y]} = \binom{[x]\wedge[y]}{[x]\wedge([X]\oplus[Y])\oplus[X]}$ wymaga czterech rund komunikacji. Przed obliczeniem $[Z] = [x] \wedge ([X] \oplus [Y]) \oplus [X]$ trzeba poznać wartość $[x] \wedge ([X] \oplus [Y])$, a wcześniej $[X] \oplus [Y]$. Jednak nie ma znaczenia, czy $[z] = [x] \wedge [y]$ zostanie wyliczone wcześniej czy później niż $[Z]$. Dane rundy komunikacji wymaganej do obliczenia $[z]$ mogą być wysłane równolegle z

danymi jednej z trzech rund komunikacji potrzebnych do obliczenia $[Z]$. Dlatego obliczenie działania $\diamond$ wymaga trzech zbiorów rund komunikacji.

Jako że działanie $\diamond$ jest łączne, wyrażenie

$$\begin{pmatrix} [z] \\ [Z] \end{pmatrix} = \begin{pmatrix} d_{l-1} \oplus [r_{l-1}] \\ d_{l-1} \end{pmatrix} \diamond \cdots \diamond \begin{pmatrix} d_0 \oplus [r_0] \\ d_0 \end{pmatrix} \diamond \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

można obliczyć w dowolnej kolejności. W pierwszym kroku można wyznaczyć wartość $\lfloor \frac{l-1}{2} \rfloor$ par $\binom{[x]}{[X]} \diamond \binom{[y]}{[Y]}$, w drugim $\lfloor \frac{l-1}{4} \rfloor$ par itd. W ten sposób zamiast $l-1$ zbiorów rund komunikacji wykonuje się $O(\log(l))$ zbiorów rund komunikacji [Bog+09, (Lemma 2.)]. Jako że obliczenie $\binom{[x]}{[X]} \diamond \binom{[y]}{[Y]}$ wymaga jednej więcej rundy komunikacji niż wyliczenie $\binom{[x]}{X} \diamond \binom{[y]}{[Y]}$, liczba rund komunikacji zwiększa się. Obliczenie l działań $\diamond$ pozostaje ograniczone przez $O(4l)$ rund komunikacji.

W pracy [Dam+06] podano sposób konstrukcji protokołu porównania o stałej, niezależnej od l liczbie zbiorów rund komunikacji. Jednak liczba bitów reprezentacji binarnej używanych w *Aukciszku* liczb nie uzasadnia użycia podobnej metody.

3.1.7. Bezpieczeństwo

Słabością protokołu porównania zastosowanego w *Aukciszku* jest jego zachowanie, kiedy losowa liczba r przyjmuje wartość bliską krańcowi przedziału. Możliwych wartości różnicy $a-b$ jest $2^{l+1}-1$. Jeżeli liczba d należy do $[2; 2^{l+1}-1]$ lub $[2^{l+k+1}+2; 2^{l+k+1}+2^{l+1}-1]$, liczba możliwych wartości $a-b$, na której podstawie powstała, jest mniejsza niż $2^{l+1}-1$. Wtedy rekonstrukcja d przybliża wartość różnicy porównywanych liczb.

Liczby a, b są zapisane na l bitach, dlatego

$$2^l + a - b \ \in \ [1;\ 2^{l+1}-1].$$

Ponieważ r jest zapisana na $l+k+1$ bitach, to

$$2^{l+k+1} - r \in [1;\ 2^{l+k+1}].$$

Z definicji liczby d:

$$d = 2^{l+k+1} - r + 2^l + a - b,$$

$$2^l + a - b = d - (2^{l+k+1} - r).$$

Dlatego jednocześnie musi być spełniony warunek

$$2^l + a - b \ \in \ [d - 2^{l+k+1};\ d-1]$$

oraz

$$2^l + a - b \in [1;\ 2^{l+1} - 1].$$

Z tego wynika, że jeżeli np. $d = 2^l$, to

$$2^l + a - b \in [2^l - 2^{l+k+1};\ 2^l - 1] \cap [1;\ 2^{l+1} - 1]$$

$$2^l + a - b \in [1;\ 2^l - 1].$$

W najgorszym przypadku, kiedy d przyjmuje minimalną/maksymalną wartość, rekonstrukcja d ujawnia dokładną wartość różnicy $a - b$. Jest tak dlatego, że minimalna/maksymalna wartość d może być uzyskana jedynie wtedy, kiedy r i $a - b$ jednocześnie przyjmą swoje minimalne/maksymalne wartości.

Prawdopodobieństwo, że wartość d pozwala na zawężenie przedziału możliwych wartości różnicy jest równe prawdopodobieństwu, że

$$r \in [2;\ 2^{l+1} - 1] \cup [2^{l+k+1} + 2;\ 2^{l+k+1} + 2^{l+1} - 1].$$

Wynosi ono

$$\frac{2 \cdot (2^{l+1} - 2)}{2^{l+k+1}} = \frac{2^{l+2} - 4}{2^{l+k+1}} \approx \frac{2^{l+2}}{2^{l+k+1}} = 2^{-(k-1)}.$$

Z tego powodu mówimy, że protokół jest statystycznie bezpieczny [Bog+09, (Lemma 2.)]. W jego zastosowaniach należy przyjąć odpowiednio dużą wartość k, aby prawdopodobieństwo wycieku przybliżenia różnicy było pomijalnie małe.

3.2. Porównanie liczb na podstawie podziałów ich bitów

Opisany w części 3.1 protokół porównania wykonuje się z prawdopodobieństwem równym 1, ale jest tylko statystycznie bezpieczny. Alternatywą jest porównanie liczb a, b na podstawie podziałów bitów, opisane w książce [CDN15, (9.1.6)].

Uczestnicy mogliby sami składać oferty w reprezentacji binarnej, ale bez rekonstrukcji udziałów nie sposób stwierdzić, czy liczba na pewno jest wartością binarną. Mogłoby to zwiększać prawdopodobieństwo błędów uniemożliwiających rozstrzygnięcie aukcji. Dlatego stosuje się protokół wyznaczenia podziałów bitów na podstawie podziału liczby. Jego częścią jest protokół generowania podziału losowej liczby z $\mathbb{Z}_p$. Prawdopodobieństwo jego powodzenia jest mniejsze niż 1, czasem należy go powtórzyć. Stąd liczba rund komunikacji wymaganych do porównania nie jest stała. Wtedy jednak protokół porównania nie niesie ryzyka ujawnienia nadmiarowych informacji o używanych liczbach.

3.2.1. Podział losowej liczby z $\mathbb{Z}_p$

Wykorzystując protokół podziału losowego bitu (protokół 3.1), można skonstruować protokół podziału liczby zbudowanej z m losowych bitów. W tym celu podzielone bity r_i są bezpiecznie mnożone przez odpowiadającą im potęgę dwójki 2^i i bezpiecznie dodane, tworząc podział losowej liczby $r = \sum_{i=0}^{m} 2^i \cdot r_i$. Żeby liczba ta mogła przyjmować dowolną wartość z $\mathbb{Z}_p$, należy użyć tyle bitów, na ilu zapisana jest liczba p, tzn. m bitów. Z tego względu jest możliwe, że bity wyznaczają liczbę większą od p. Jako że nie można wyznaczyć podziału takiej liczby, należy powtórzyć protokół. Żeby rozpoznać taką sytuację, wykonywane jest bezpieczne porównanie bitów r_i z bitami liczby p.

Protokół 3.4. Generowanie podziału losowej liczby z $\mathbb{Z}_p$ i podziałów jej bitów [CDN15, (9.1.6 Protocol RandomSolvedBits)].
Dane:
- p - publicznie znana liczba pierwsza,
- m - liczba bitów reprezentacji binarnej liczby p,
- $p_{m-1}, \ldots, p_0$ - bity p w kolejności od najbardziej znaczącego bitu.

Przebieg:
1. Strony m razy wykonują protokół podziału losowego bitu, otrzymując podziały bitów $[r_{m-1}], \ldots, [r_0]$.
2. Strony bezpiecznie porównują liczbę p z liczbą r na podstawie p_i oraz $[r_i]$.
3. Jeżeli $p > r$, strony powtarzają protokół.
4. Jeżeli $p \leqslant r$, strony obliczają podział liczby r, wykonując bezpieczne dodawanie i mnożenie przez stałą:

$$[r] = \sum_{i=0}^{m-1} 2^i \cdot [r_i].$$

Wyjście:
- $[r]$ - podział wartości zmiennej losowej r o wartościach ze zbioru $\mathbb{Z}_p$ i rozkładzie jednostajnym,
- $[r_{m-1}], \ldots, [r_0]$ - podziały bitów r w kolejności od najbardziej znaczącego bitu.

Bity r_i są losowane z rozkładu jednostajnego, więc r również ma rozkład jednostajny i przyjmuje wartości ze zbioru $\{0, \ldots, 2^m - 1\}$. Jeżeli r jest większa od p, protokół jest powtarzany. Dlatego r jest zmienną losową o wartościach ze zbioru $\mathbb{Z}_p$ i rozkładzie jednostajnym.

Liczba bitów reprezentacji binarnej p to m, więc

$$2^{m-1} \leqslant p < 2^m.$$

Prawdopodobieństwo powodzenia protokołu wynosi co najmniej $\frac{1}{2}$.

$$\mathbb{P}(p \leqslant r) \geqslant \frac{2^{m-1}}{2^m} = 2^{-1}$$

Protokół wymaga wylosowania m podziałów bitów $[r_i]$ oraz bezpiecznego porównania bitów r_i z bitami liczby p. Opisany poniżej protokół 3.5 porównania bitów wymaga $3m - 1$ rund komunikacji. Dlatego wygenerowanie podziału losowej liczby z $\mathbb{Z}_p$ i podziałów jej bitów wymaga $4m - 1$ rund komunikacji.

3.2.2. Porównanie liczb na podstawie podziałów ich bitów

Zakładamy, że dysponujemy podziałami bitów reprezentacji binarnej liczb a, b w kolejności od najbardziej znaczącego bitu numer $m - 1$. Celem protokołu jest otrzymanie nowego podziału sekretu, który będzie ukrywał wartość 1, jeżeli $a > b$, albo 0, jeżeli $a \leqslant b$.

Protokół 3.5. Porównanie liczb na podstawie podziałów ich bitów [CDN15, (9.1.6 Protocol BitCompare, Protocol MS1)].
Parametry:
— m - liczba bitów reprezentacji binarnej porównywanych liczb.
Wejście:
— $[a_{m-1}], \ldots, [a_0]$ - podziały bitów liczby a,
— $[b_{m-1}], \ldots, [b_0]$ - podziały bitów liczby b.
Przebieg:

1. Strony wykonują bezpieczny XOR wszystkich bitów a_i i b_i, definiując podziały bitów c_i. Dla $i = m - 1, \ldots, 0$:

$$[c_i] = [a_i] \oplus [b_i].$$

2. Strony wykonują bezpieczne dodawanie i mnożenie przez stałą bitów, definiując podziały bitów f_i. Aby rozpocząć obliczenia, za $[f_m]$ przyjmują stałą wartość 1. Dla $i = m - 1, \ldots, 0$:

$$[f_i] = [f_{i+1}] \cdot (1 - [c_i]).$$

3. Strony wykonują bezpieczne odejmowanie, otrzymując podziały bitów d_i. Dla $i = m - 1, \ldots, 0$:

$$[d_i] = [f_{i+1}] - [f_i].$$

4. Strony wykonują bezpieczne mnożenie, definiując podziały bitów e_i. Dla $i = m - 1, \ldots, 0$:

$$[e_i] = [a_i] \cdot [d_i].$$

5. Strony wykonują bezpieczne dodawanie, otrzymując podział bitu w, oznaczającego wynik porównania:

$$[w] = [e_{m-1}] + \ldots + [e_0].$$

Wyjście:
— $[w]$ - podział bitu oznaczającego wynik porównania. Jeżeli $a > b$, wtedy $w = 1$. W przeciwnym przypadku $w = 0$.

Jeżeli zachodzi potrzeba zmiany protokołu, aby zwracał podział wartości 1, kiedy liczba a jest większa lub równa b, można powiększyć reprezentację binarną liczb a, b o odpowiednio spreparowany bit na najmniej znaczącej pozycji. Zmiana nie wpływa na liczbę rund komunikacji.

Przykład 3.1. Porównanie liczb na podstawie udziałów ich bitów. Celem jest otrzymanie nowego podziału sekretu, który będzie ukrywał wartość 1, jeżeli $a \geqslant b$, albo 0, jeżeli $a < b$.
Parametry:
— $m = 4$,
— $a = 11$,
— $b = 10$.
Wejście:
— $[1], [0], [1], [1]$ - podziały bitów liczby a,
— $[1], [0], [1], [0]$ - podziały bitów liczby b.
Przebieg:
1. Aby protokół zwrócił podział bitu 1, kiedy liczby są równe, strony ustalają dodatkowe bity na najmniej znaczącej pozycji ($a_0 = 1$, $b_0 = 0$). Są to stałe, a nie podziały sekretu, dlatego nie wpływają na liczbę rund komunikacji.
   — $\{[a_4], [a_3], [a_2], [a_1], a_0\} = \{[1], [0], [1], [1], 1\}$
   — $\{[b_4], [b_3], [b_2], [b_1], b_0\} = \{[1], [0], [1], [1], 0\}$
2. Protokół opiera się na znalezieniu najbardziej znaczącego bitu, na którym liczby się różnią. Bit większej z liczb na tej pozycji będzie równy 1. Żeby sprawdzić, czy bity się różnią, obliczany jest ich XOR.
   $[c_i] = [a_i] \oplus [b_i]$
   — $[c_4] = [1] \oplus [1] = [0]$
   — $[c_3] = [0] \oplus [0] = [0]$
   — $[c_2] = [1] \oplus [1] = [0]$
   — $[c_1] = [1] \oplus [0] = [1]$
   — $[c_0] = 1 \oplus 0 = 1$
3. Następnie, począwszy od najbardziej znaczącego bitu, pozycje na których bity są takie same oznaczane są liczbą 1. Począwszy od pierwszej pozycji, na której bity się różnią, wszystkie następne pozycje oznaczane są zerami.

Oznaczenia zapisywane są jako podziały $[f_i]$. Aby rozpocząć obliczenia, za $[f_5]$ przyjmuje się stałą 1.
$[f_i] = [f_{i+1}] \cdot (1 - [c_i])$
— $[f_4] = 1 \cdot (1 - [0]) = [1]$
— $[f_3] = [1] \cdot (1 - [0]) = [1]$
— $[f_2] = [1] \cdot (1 - [0]) = [1]$
— $[f_1] = [1] \cdot (1 - [1]) = [0]$
— $[f_0] = [0] \cdot (1 - 0) = [0]$

4. Pierwsza pozycja, na której liczby się różnią, to jedyna pozycja oznaczona zerem, dla której pozycja przed nią jest oznaczona jedynką. Tylko wtedy $f_{i+1} - f_i$ jest równe 1.
$[d_i] = [f_{i+1}] - [f_i]$
— $[d_4] = 1 - [1] = [0]$
— $[d_3] = [1] - [1] = [0]$
— $[d_2] = [1] - [1] = [0]$
— $[d_1] = [1] - [0] = [1]$
— $[d_0] = [0] - 0 = [0]$
5. Wszystkie pozycje, na których $d_i = 0$, przemnożone przez a_i dadzą wynik 0. Wśród e_i pojawi się jedynka tylko wtedy, gdy na pozycji, na której bity się różnią ($d_i = 1$), liczba a jest większa od b ($a_i = 1$).
$[e_i] = [a_i] \cdot [d_i]$
— $[e_4] = [1] \cdot [0] = [0]$
— $[e_3] = [0] \cdot [0] = [0]$
— $[e_2] = [1] \cdot [0] = [0]$
— $[e_1] = [1] \cdot [1] = [1]$
— $[e_0] = 1 \cdot [0] = [0]$
6. Jeżeli wśród e_i znajduje się jedynka, suma bitów e_i będzie wynosić 1. W przeciwnym przypadku będzie zerem. Dlatego poprawnie koduje ona wynik porównania.
$w = e_4 + \ldots + e_0$
$[w] = [0] + [0] + [0] + [1] + [0] = [1]$

Protokół wymaga obliczenia m wyrażeń $[a_i] \oplus [b_i]$, $m - 1$ bezpiecznych iloczynów $[f_{i+1}] \cdot (1 - [c_i])$ oraz m iloczynów $[a_i] \cdot [d_i]$. Dlatego jedno porównanie liczb na podstawie ich bitów wymaga $3m - 1$ rund komunikacji.

3.2.3. Wyznaczenie podziałów bitów liczby

Korzystając z powyższych protokołów, można wyznaczyć podziały bitów liczby na podstawie jej podziału. Od podziału liczby a odejmuje się podział losowej

liczby r, otrzymując pewną liczbę oznaczaną jako c. Tę liczbę można zrekonstruować bez ryzyka ujawnienia żadnych informacji o a, o ile nie rekonstruuje się r ani bitów r_i. Ze względu na redukcję modulo będzie ona równa $a - r$ albo $p + (a - r)$, jeżeli $a - r < 0$. Po rekonstrukcji c można jawnie wyznaczyć bity liczby c, które zostaną dodane do podzielonych bitów r_i. W ten sposób otrzymuje się podziały bitów liczby d równej a albo $p + a$, jeżeli $d \geqslant p$. Jeżeli d jest większe od p, od bitów d_i odejmuje się bity p_i.

Protokół 3.6. Wyznaczenie podziałów bitów liczby na podstawie jej podziału [CDN15, (9.1.6 Protocol Bits)].
Dane:
— p - liczba pierwsza,
— l - liczba bitów reprezentacji binarnej porównywanych liczb,
— m - liczba bitów reprezentacji binarnej liczby p.
Wejście:
— $[a]$ - podział liczby a,
— $[r_{m-1}], \ldots, [r_0]$ - podziały bitów losowej liczby r,
— $[r]$ - podział losowej liczby r.
Przebieg:

1. Strony wykonują bezpieczne odejmowanie, otrzymując podział $[c]$.

$$[c] = [a] - [r]$$

2. Strony dokonują rekonstrukcji wartości c. Nie ujawnia ona żadnych informacji o a, o ile nie nastąpi rekonstrukcja liczby r. Wszystkie strony poznają wartość c i wyznaczają jej bity.

$$c_{m-1}, \ldots, c_0$$

3. Strony bezpiecznie dodają liczby c i r na podstawie ich bitów. Jako wynik otrzymują podziały bitów liczby d, równej a albo $p + a$, jeżeli $r > a$.

$$[d_m], [d_{m-1}], \ldots, [d_0]$$

4. Strony wykonują bezpieczne porównanie liczb d i p na podstawie podziałów ich bitów. Jako wynik otrzymują podział bitu oznaczany $[e]$.

$$d \geqslant p \;\Rightarrow\; e = 1$$

$$d < p \;\Rightarrow\; e = 0$$

5. Strony bezpiecznie odejmują od liczby d liczbę $e \cdot p$ na podstawie podziałów ich bitów ($[d_i]$ i $[e] \cdot p_i$). Zarówno kiedy d jest równe a, jaki i kiedy wynosi $p + a$, jako wynik otrzymują podziały bitów a.

$$d - e \cdot p = a$$

$$[a_{m-1}], \ldots, [a_0]$$

Wyjście:

— $[a_m], \ldots, [a_0]$ - podziały bitów liczby a.

Dodawanie bitów

Dodawanie liczb c, r na podstawie bitów $c_{m-1}, \ldots, c_0$ i podziałów $[r_{m-1}], \ldots, [r_0]$ jest wykonywane od najmniej znaczącego bitu z zachowaniem bitu przeniesienia. W kroku i oblicza się podział bitu sumy $[s_i]$ i podział przeniesienia $[g_i]$. Aby rozpocząć obliczenia, za $[g_{-1}]$ przyjmuje się stałą 0.

$$[s_i] \leftarrow c_i \oplus [r_i] \oplus [g_{i-1}]$$

$$[g_i] \leftarrow c_i \cdot [r_i] + (c_i \oplus [r_i]) \cdot [g_{i-1}]$$

Obliczenia wymagające komunikacji to XOR $(c_i \oplus [r_i]) \oplus [g_{i-1}]$ oraz mnożenie $(c_i \oplus [r_i]) \cdot [g_{i-1}]$, gdzie oba czynniki działania są podziałami. Ponieważ $x \oplus y$ jest obliczane jako $x + y - 2xy$, wynik mnożenia $(c_i \oplus [r_i]) \cdot [g_{i-1}]$ może być wykorzystany do obliczenia $(c_i \oplus [r_i]) \oplus [g_{i-1}]$. Dlatego dodanie pary bitów wymaga jednej rundy komunikacji.

Negacja bitu

Negację podzielonego bitu x można obliczyć jako $-1 \cdot [x] + 1$, wykorzystując bezpiecznie dodawanie i mnożenie sekretu przez stałą.

x	$-1 \cdot x$	$-1 \cdot x + 1$
0	0	1
1	-1	0

Negacja bitu nie wymaga komunikacji pomiędzy stronami.

Odejmowanie bitów

Niech $f = e \cdot p_i$. Różnicę $d - f$ na podstawie podziałów $[d_m], \ldots, [d_0]$ i $[f_{m-1}], \ldots, [f_0]$ oblicza się od najmniej znaczącego bitu z zachowaniem bitu przeniesienia. W kroku i oblicza się podział bitu różnicy $[s_i]$ i podział przeniesienia $[g_i]$. Aby rozpocząć obliczenia, za $[g_{-1}]$ przyjmuje się stałą 0.

$$[s_i] \leftarrow [d_i] \oplus [f_i] \oplus [g_{i-1}]$$

$$[g_i] \leftarrow \neg[d_i] \cdot [g_i] + \neg[d_i] \cdot [f_i] + [f_i] \cdot [g_{i-1}]$$

Wynik mnożenia $[f_i] \cdot [g_{i-1}]$ może być wykorzystany do obliczenia $[f_i] \oplus [g_{i-1}]$. Dlatego odejmowanie pary bitów wymaga czterech rund komunikacji.

Wymagana liczba rund komunikacji

Przed rozpoczęciem obliczeń wymagane jest wyznaczenie podziału losowej liczby i podziałów jej m bitów. Jako że prawdopodobieństwo jego powodzenia nie jest pewne, oznacza to co najmniej $4m - 1$ rund komunikacji. Dodatkowo wykonywane jest dodawanie liczb zapisanych na m bitach i odejmowanie liczb zapisanych na $m + 1$ bitach, wymagające odpowiednio m i $4(m + 1)$ rund komunikacji. Wykonywane jest też bitowe porównanie d z p. Jako że część mnożeń sekretów jest zastępowana mnożeniami sekretu przez stałą, takie porównanie jest mniej kosztowne niż porównanie dwóch liczb danych podziałami bitów. Wymaga ono $m - 1$ rund komunikacji. Stąd wyznaczenie podziałów bitów liczby na podstawie jej podziału wymaga co najmniej $10m + 2$ rund komunikacji pomiędzy stronami.

Rozdział 4

Implementacja projektu

4.1. Implementacja systemu aukcyjnego

W tej części pokrótce opiszemy przebieg aukcji na platformie *Aukciszek*, zastosowanie protokołów kryptograficznych i innych zabezpieczeń.

4.1.1. Strony aukcji

Stronami aukcji są składający oferty uczestnicy, podmioty pośredniczące, przeprowadzające porównanie w celu rozstrzygnięcia aukcji, oraz administrator, który nadzoruje proces aukcyjny. Administrator obsługuje część systemu przeznaczoną dla uczestników, zarządzając rejestracją na aukcję. Podmioty pośredniczące uruchamiają na udostępnianych przez siebie serwerach obliczeniowych system obliczeń wielostronnych. Zarejestrowani uczestnicy mogą składać oferty, wysyłając je na serwery, i dowiedzieć się o wyniku aukcji od administratora. Administrator koordynuje komunikację serwerów, które wspólnie porównują oferty i ujawniają ofertę wygrywającą.

Aplikacja internetowa

Część systemu przeznaczona dla uczestników to aplikacja internetowa zbudowana z użyciem technologii `Next.js` i używająca bazy danych `Supabase`. Służy do rejestracji użytkowników na aukcję, składania ofert i ogłoszenia wyników aukcji. Jest obsługiwana przez administratora, który korzysta z niej również w celu wydawania poleceń serwerom obliczeniowym. Komunikacja jest zabezpieczona protokołem `HTTPS`.

Uwierzytelnianie użytkowników i serwerów

Administrator, aby wydawać polecenia wykonania obliczeń serwerom, oraz użytkownicy, żeby móc wysłać udziały swoich ofert na serwery, muszą być uwierzytelnieni przez wyróżniony serwer dystrybuujący tokeny `JWT`. Komunikacja między użytkownikami a serwerami jest zabezpieczone protokołem `HTTPS`.

4.1.2. Składanie ofert

Strony ustalają przedział liczbowy możliwych wysokości ofert i parametry bezpieczeństwa wpływające na czas obliczeń. Uczestnik wprowadza wysokość swojej oferty w systemie, który wyznacza jej udziały. Używany jest schemat podziału sekretu Shamira z progiem rekonstrukcji $t = \lfloor \frac{n-1}{2} \rfloor$. Proces wyznaczania udziałów odbywa się na komputerze uczestnika, dlatego oferta pozostaje tajna. Uczestnik wysyła na każdy z serwerów obliczeniowych przeznaczony dla niego udział. Połączenie użytkownika z serwerem jest zabezpieczona protokołem `HTTPS`.

Wyznaczenie udziałów przez uczestników i wysłanie ich na serwery obliczeniowe odbywa się asynchronicznie, w okresie od ogłoszenia aukcji do terminu zakończenia przyjmowania ofert.

4.1.3. Rozstrzygnięcie aukcji

Po zakończeniu przyjmowania ofert administrator inicjuje obliczenia wielostronne, służące do wyboru najlepszej oferty. Używany algorytm porównania pozwala stwierdzić, czy dana liczba jest większa lub równa drugiej, bez ujawniania różnicy między liczbami.

Porównanie ofert wymaga synchronicznej komunikacji serwerów. Administrator nie ma kontroli nad serwerami ani wglądu w ich udziały. Koordynuje jednak ich komunikację i wydaje polecenia wykonywania obliczeń. Poznaje także rekonstruowane wartości pośrednie, w tym wyniki poszczególnych porównań.

Aplikacja serwerowa

Pośrednicy uruchamiają na swoich serwerach obliczeniowych aplikację napisaną z wykorzystaniem `Python FastAPI`. Serwery identyfikują się nawzajem za pomocą publicznych adresów `IP`. Ich wzajemna komunikacja może być zabezpieczona protokołem `HTTPS` lub przez tunelowanie `WireGuard`.

4.1.4. Porównanie ofert

Kiedy liczba ofert to n, rozstrzygnięcie aukcji odbywa się poprzez $n-1$ porównań ofert, z zachowaniem informacji o aktualnie najkorzystniejszej. Jako że tożsamości uczestników są znane a oferty rozróżnialne, może to zdradzać pewne niepotrzebne informacje. W najgorszym przypadku porównanie wyznaczy porządek liniowy zbioru ofert.

Aby wykryć sytuację, w której kilka ofert jest równych wygrywającej, można przeprowadzić porównanie w odwrotnej kolejności. Postępowanie w przypadku remisu może się różnić w zależności od celu i założeń aukcji.

Porównanie z zachowaniem informacji o aktualnie najkorzystniejszej ofercie wymaga rekonstrukcji wyniku każdego porównania. Może to wyznaczyć porządek liniowy zbioru ofert. Chcąc wyeliminować tę ewentualność, można użyć metody rozstrzygnięcia aukcji bez rekonstrukcji wyników poszczególnych porównań.

Porównanie bez rekonstrukcji

Niech $[o_1], [o_2], \ldots, [o_n]$ to podziały wysokości ofert. Niech $o_{i,j}$ to bit oznaczający wynik porównania o_i z o_j, dla $i, j = 1, \ldots, n$. Bit $o_{i,j}$ jest równy 1, kiedy $o_i \geqslant o_j$, w przeciwnym wypadku przyjmuje wartość zero. Używając bezpiecznego protokołu porównania sekretów, można obliczyć podział $[o_{i,j}]$ na podstawie podziałów $[o_i], [o_j]$. Kiedy $i = j$, zamiast podziału można przyjąć stałą wartość 1.

$[o_{i,j}] \leftarrow [o_i \geqslant o_j]$	$[o_1]$	$[o_2]$	$\cdots$	$[o_n]$	$[o_{i,1} \cdot o_{i,2} \cdot \ldots \cdot o_{i,n}]$
$[o_1]$	1	$[o_{1,2}]$	$\cdots$	$[o_{1,n}]$	$1 \cdot [o_{1,2}] \cdot \ldots \cdot [o_{1,n}]$
$[o_2]$	$[o_{2,1}]$	1	$\cdots$	$[o_{1,n}]$	$[o_{2,1}] \cdot 1 \cdot \ldots \cdot [o_{2,n}]$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$[o_n]$	$[o_{n,1}]$	$[o_{n,2}]$	$\cdots$	1	$[o_{n,1}] \cdot [o_{n,2}] \cdot \ldots \cdot 1$

Tabela 4.1. Podziały oznaczające wyniki porównań. Rekonstruowane są tylko rezultaty działań z ostatniej kolumny.

Iloczyn $o_{i,1} \cdot o_{i,2} \cdot \ldots \cdot o_{i,n}$ przyjmie wartość 1 wtedy i tylko wtedy, kiedy wszystkie jego czynniki będą równe 1, tzn. oferta numer i będzie wyższa lub równa każdej innej ofercie. Obliczając podział takiego iloczynu dla wszystkich wartości $i = 1, \ldots, n$, a następnie rekonstruując je, można wyznaczyć wygrywającą ofertę. Jeżeli kilka ofert jest równych wygrywającej, będą znane ich indeksy. Ponieważ wartości $o_{i,j}$ nie muszą być rekonstruowane, nie zostaną ujawnione żadne zależności pomiędzy przegrywającymi ofertami.

Ta metoda rozstrzygnięcia aukcji wymaga $n \cdot (n-1)$ porównań i $n \cdot (n-2)$ dodatkowych iloczynów sekretów. Obecnie nie jest wdrożona w systemie *Aukciszek*.

4.1.5. Ogłoszenie wygrywającej oferty

Nikt, nawet administrator, nie może zmusić serwerów do rekonstrukcji sekretu użytkownika. Operatorzy ponad połowy serwerów muszą samodzielnie zgodzić się na ogłoszenie wysokości wygrywającej oferty.

Załóżmy, że spośród grupy rekonstruującej sekret wszyscy pośrednicy wyjawią swoje udziały poza jednym. Wtedy jako jedyny pozna on wartość sekretu. Istnieją sposoby asynchronicznego zbierania udziałów, które z wysokim prawdopodobieństwem zapewniają, że dokona się rekonstrukcji poprawnego sekretu, a podmiot przesyłający swój udział jako ostatni nie pozna sekretu, jeżeli nie ujawni poprawnie swojego udziału [HLL15].

Jednak założenia *Aukciszka* sprawiają, że nie wszystkie udziały muszą być użyte do rekonstrukcji wygrywającej oferty. Założenie występowania zaufanej większości wystarcza, żeby uczciwe podmioty poznały wynik aukcji.

Wszystkie udziały są potrzebne do wykonywania mnożenia sekretów. Jednak po ostatnim mnożeniu wymagane jest dodawanie, do powodzenia którego większość stron musi mieć poprawne udziały. Stąd grupa licząca mniej niż połowę pośredników, podając fałszywe udziały podczas mnożenia, nie osiągnie nic poza zablokowaniem aukcji.

4.2. Testy i zakończenie projektu

Testy projektu wykonaliśmy przez uruchomienie serwerów na platformie `Microsoft Azure` i przeprowadzanie aukcji z różnymi liczbami ofert i parametrów bezpieczeństwa. Po prezentacji i złożeniu dokumentacji projekt został zakończony z wysoką oceną. Spełniliśmy założenia dotyczące przebiegu aukcji, bezpieczeństwa i wydajności. Możliwe jest usprawnienie projektu w zakresie bezpieczeństwa i wydajności oraz wprowadzenie dodatkowych funkcjonalności, z których część opisano w tej pracy.

Bibliografia

[Sha79] Adi Shamir. "How to share a secret". In: *Communications of the ACM* 22.11 (1979), pp. 612–613.

[WOG88] Avi Wigderson, MB Or, and S Goldwasser. "Completeness theorems for noncryptographic fault-tolerant distributed computations". In: *Proceedings of the 20th Annual Symposium on the Theory of Computing (STOC'88)*. 1988, pp. 1–10.

[VMV96] Paul C Van Oorschot, Alfred J Menezes, and Scott A Vanstone. *Handbook of applied cryptography*. CRC press, 1996.

[Dam+06] Ivan Damgård et al. "Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation". In: *Theory of Cryptography Conference*. Springer. 2006, pp. 285–304.

[Bog+09] Peter Bogetoft et al. "Secure multiparty computation goes live". In: *International Conference on Financial Cryptography and Data Security*. Springer. 2009, pp. 325–343.

[CDN15] Ronald Cramer, Ivan Bjerre Damgård, and Jesper Buus Nielsen. *Secure multiparty computation and secret sharing*. Cambridge University Press, 2015.

[HLL15] Lein Harn, Changlu Lin, and Yong Li. "Fair secret reconstruction in (t, n) secret sharing". In: *Journal of Information Security and Applications* 23 (2015), pp. 1–7.

[Sma16] Nigel P Smart. *Cryptography made simple*. Springer, 2016.

[BS23] Dan Boneh and Victor Shoup. *A graduate course in applied cryptography*. 2023.